IF4031 Arsitektur Aplikasi Terdistribusi

Tugas Besar – Milestone 1: Sistem Terdistribusi Dasar

# ARGUS

## Aggregated Real-time Geohazard Unified System

Gateway koordinasi kebencanaan BNPB yang menyatukan data BMKG dan PVMBG


| | |
|---|---|
| **Nama kelompok** | OrangSuksesAamiin |
| **Anggota** | Andi Farhan Hidayat NIM 13523128 |
| | Muhammad Rizain Firdaus NIM 13523164 |
| | Muhammad Kinan Arkansyaddad NIM 13523152 |


| | |
|---|---|
| **Mata kuliah** | IF4031 Arsitektur Aplikasi Terdistribusi |
| **Institusi** | Institut Teknologi Bandung |
| **Semester** | 7, 2026/2027 |
| **Tenggat** | Sabtu, 10 Oktober 2026, 23.59 WIB |

Berkas ini dikumpulkan sebagai `IF4031_M1_<NamaKelompok>.pdf`. Bagian bertanda **[merah]** masih harus diisi oleh kelompok.

# Daftar Isi


**1 Bagian Umum** **1**
- 1.1 Tabel kontribusi . . . 1
- 1.2 Deskripsi umum sistem . . . 1
- 1.3 Diagram arsitektur . . . 2
- 1.4 Teknologi terpilih . . . 4
- 1.5 Asumsi . . . 4
- 1.6 Petunjuk menjalankan . . . 5
- 1.7 Penggunaan LLM . . . 5
- 1.8 Istilah penting . . . 5

**2 Problem 1 – Arsitektur dan Interoperabilitas Data** **7**
- 2.1 Rancangan solusi dan mekanisme . . . 7
  - 2.1.1 Aturan pemetaan . . . 8
  - 2.1.2 Jenis perubahan skema yang didukung dan tidak . . . 8
- 2.2 Alasan keputusan dan trade-off . . . 8
- 2.3 Batasan yang disadari . . . 10
- 2.4 Bukti penyelesaian . . . 10
- 2.5 Jawaban butir “Di laporan” . . . 11

**3 Problem 2 – Concurrency, Availability, dan Graceful Degradation** **13**
- 3.1 Rancangan solusi dan mekanisme . . . 13
  - 3.1.1 Bagaimana model ini mencegah tiga kegagalan . . . 14
- 3.2 Alasan keputusan dan trade-off . . . 14
- 3.3 Batasan yang disadari . . . 14
- 3.4 Konfigurasi dan tooling load test . . . 15
- 3.5 Bukti penyelesaian . . . 15
- 3.6 Jawaban butir “Di laporan” . . . 17

**4 Problem 3 – Autentikasi dan Kepercayaan antar-Service** **18**
- 4.1 Rancangan solusi dan mekanisme . . . 18
  - 4.1.1 Klien downstream: Media, Tim Lapangan, BNPB Pusat . . . 18
- 4.2 Alasan keputusan dan trade-off . . . 19
- 4.3 Batasan yang disadari . . . 20
- 4.4 Bukti penyelesaian . . . 20
- 4.5 Jawaban butir “Di laporan” . . . 22

**5 Problem 4 – Arsitektur Microservice dan Flexible Storage** **23**
- 5.1 Rancangan solusi dan mekanisme . . . 23
  - 5.1.1 Pembagian service . . . 23
  - 5.1.2 Aturan kepemilikan penyimpanan . . . 24
  - 5.1.3 Penyimpanan yang fleksibel: PostgreSQL dengan JSONB . . . 24
  - 5.1.4 Isolasi jaringan sebagai bukti kepemilikan . . . 24

5.2 Alasan keputusan dan trade-off . . . . . 24
5.2.1 Batas service: mengapa dipecah seperti ini . . . . . 24
5.2.2 Pilihan penyimpanan . . . . . 25
5.3 Batasan yang disadari . . . . . 26
5.4 Bukti penyelesaian . . . . . 26
5.5 Jawaban butir “Di laporan” . . . . . 28

**6 Problem 5 – Fan-out Event Distribution (Pub/Sub) 29**
6.1 Rancangan solusi dan mekanisme . . . . . 29
6.1.1 Sisi producer: transactional outbox . . . . . 29
6.1.2 Sisi consumer . . . . . 30
6.1.3 Semantik pengiriman: at-least-once . . . . . 30
6.2 Alasan keputusan dan trade-off . . . . . 30
6.3 Batasan yang disadari . . . . . 31
6.4 Bukti penyelesaian . . . . . 32
6.5 Jawaban butir “Di laporan” . . . . . 33

**A Penelusuran Klaim ke Bukti 34**
A.1 Ketentuan implementasi lintas problem . . . . . 34
A.2 Matriks kriteria, perintah, dan bukti . . . . . 34
A.3 Perintah pemeriksaan penuh . . . . . 35

**B Lampiran Form Penggunaan AI 37**

# 1 Bagian Umum

## 1.1 Tabel kontribusi

Tabel ini menunjukkan siapa mengerjakan apa. Pembagian kerja mengikuti tiga tahap: fondasi dan Problem 1, lalu Problem 2 dan 3, lalu Problem 4 dan 5 beserta integrasi.

| Anggota | Perancangan | Implementasi | Penulisan laporan |
|---|---|---|---|
| Orang 1<br>Andi Farhan Hidayat | Kontrak antar-komponen, arsitektur Aggregator, kebijakan skema | Fondasi (`pkg/obs`, Compose, Makefile, CI), mock BMKG dan PVMBG, Aggregator, Problem 1, ketahanan ingest, adapter PostgreSQL dan Kafka, outbox dan relay, consumer | Bagian umum, Problem 1, deskripsi mock |
| Orang 2<br>Muhammad Rizain Firdaus | Autentikasi JWT tiga peran, kebijakan field Ringkasan/Mentah, pembatas laju, penanda `stale` | `auth-service`, `client-api`, skrip demo dan load test Problem 2 dan 3 | Problem 2, Problem 3 |
| Orang 3<br>Muhammad Kinan Arkansyaddad | Perbandingan pilihan penyimpanan (PostgreSQL dengan JSONB) dan pilihan penyebaran event (Kafka dengan outbox), semantik pengiriman, dan batasan yang disadari | Skrip demo Problem 4 dan 5 (membangun ulang satu service, field baru tanpa migrasi, consumer baru) dan pengujian pemulihan consumer | Problem 4, Problem 5 |

**Tabel 1.1:** Kontribusi anggota.

## 1.2 Deskripsi umum sistem

Di cerita soal, Astolfo menerima sinyal bencana dari banyak lembaga yang tidak saling bicara. Sistem ini, ARGUS (*Aggregated Real-time Geohazard Unified System*), adalah *gateway* milik BNPB yang menyambungkannya. Ia melakukan lima hal:

1. **Mengambil** data dari dua lembaga secara berkala (*polling*): BMKG untuk gempa dan tsunami, PVMBG untuk gunung api.
2. **Menyeragamkan** dua format yang berbeda menjadi satu format milik BNPB, yaitu *HazardEvent*.
3. **Menyimpan** hasilnya di satu penyimpanan, *Canonical Store*, yang hanya dikelola oleh satu service.
4. **Menyebarkan** setiap data baru ke pihak yang berlangganan lewat *message broker*.
5. **Menyajikan** data ke tiga jenis klien (Media, Tim Lapangan, BNPB Pusat) dengan hak akses yang berbeda.

**Lingkup Milestone 1** dibatasi oleh soal: dua lembaga sumber (BMKG dan PVMBG), dua jenis bencana (seismik dan vulkanik), BNPB sebagai simpul koordinasi, dan semua klien hanya membaca (*read-only*). BMKG dan PVMBG adalah *mock* buatan kelompok yang mengikuti kontrak di soal tanpa mengubahnya.

#### Status penyelesaian

Tabel berikut merangkum apa yang sudah terbukti lewat demo. Bukti P1 dan sisi ingest P2 dibuat pada 7 Oktober 2026; bukti sisi layanan P2, P3, P4, dan P5 pada 8 Oktober 2026. Demo P2 dan P3 dijalankan sekali setelah `make reset`. Demo P4 dan P5 dijalankan setelah reset dan diulang sekali tanpa reset; keduanya lulus kedua kali.

| Problem | Yang sudah terbukti / yang masih kurang | Status |
|---|---|---|
| P1 Interoperabilitas data | Pemetaan benar, skema PVMBG berubah saat berjalan tanpa restart, field baru masuk `attributes`, perubahan yang tidak didukung dikarantina. | **Lulus** |
| P2 Konkurensi dan ketersediaan | p95 BMKG-only di bawah 300 ms saat PVMBG 3 detik, load test 50 koneksi selama 60 detik, outage PVMBG ditangani, pulih tanpa restart. | **Lulus** |
| P3 Autentikasi dan kepercayaan | Kredensial antar-lembaga saling menolak, Media hanya Ringkasan, token Lapangan kedaluwarsa lalu di-*refresh*, tidak ada kredensial ter-commit. | **Lulus** |
| P4 Microservice dan storage | Satu container (`pvmbg-mock`) dihentikan, dibangun ulang, dan dijalankan lagi tanpa menyentuh yang lain; field baru masuk JSONB tanpa migrasi dan record lama tetap terbaca; Canonical Store hanya terjangkau oleh Aggregator. | **Lulus** |
| P5 Pub/sub | Event lewat Kafka, dua consumer independen, satu consumer dimatikan lalu menyusul event yang tepat, dan consumer ketiga ditambahkan saat sistem berjalan tanpa mengubah producer. | **Lulus** |

**Tabel 1.2:** Status per problem pada saat laporan ini ditulis.

## 1.3 Diagram arsitektur

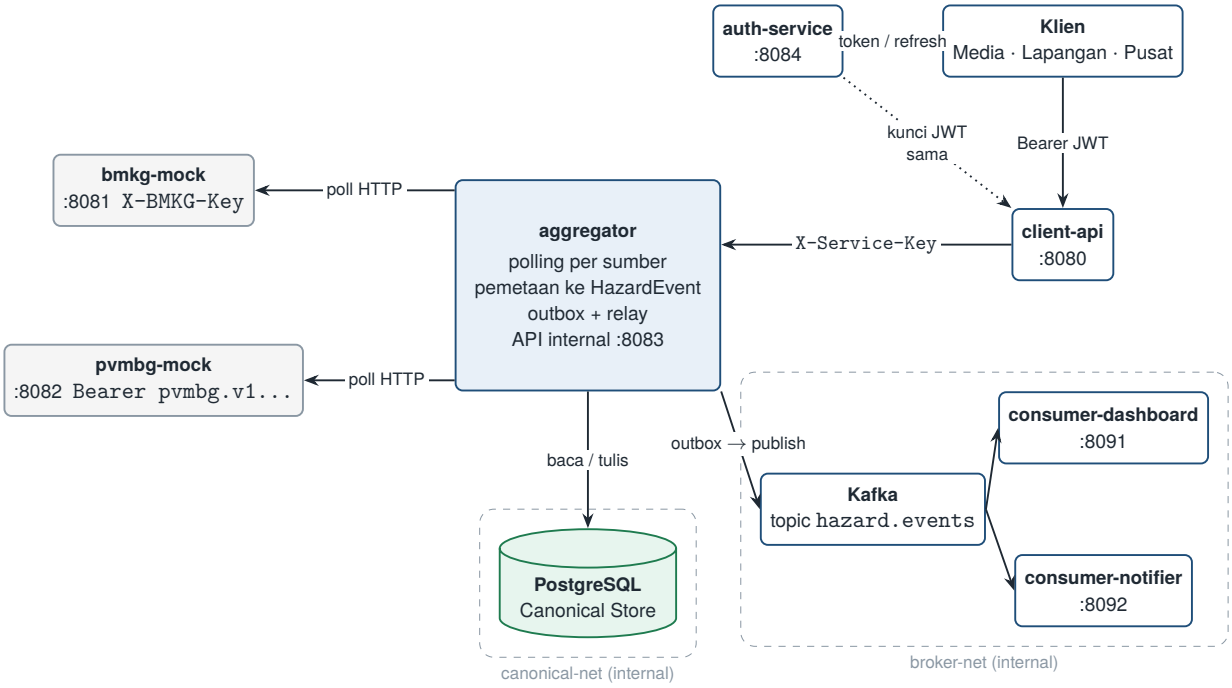
Gambar 1.1 memperlihatkan seluruh service, protokol, kepemilikan penyimpanan, dan batas jaringan.

#### Penjelasan

**Alur masuk.** `aggregator` memanggil `bmkg-mock` dan `pvmbg-mock` lewat HTTP. Kedua mock pasif: mereka hanya menjawab, tidak mengirim data sendiri. Setiap sumber punya putaran polling sendiri, sehingga PVMBG yang lambat tidak menahan BMKG (Problem 2). Hasil polling dipetakan ke *HazardEvent* dan disimpan.

**Penyimpanan.** Canonical Store adalah PostgreSQL yang hanya berada di jaringan `canonical-net`. Jaringan itu hanya berisi `aggregator` dan database-nya, jadi service lain secara fisik tidak bisa menyambung ke database (Problem 4). Service lain memperoleh data lewat API internal Aggregator.

**Alur keluar ke klien.** Klien meminta token ke `auth-service`, lalu memanggil `client-api` dengan token itu. `client-api` memeriksa token, menentukan field apa yang boleh dilihat peran tersebut, dan meneruskan permintaan ke API internal Aggregator memakai `X-Service-Key`.




**Gambar 1.1:** Arsitektur sistem. Garis tegas: panggilan jaringan; garis titik: rahasia penanda tangan token yang dipegang bersama. Kotak putus-putus: jaringan Docker yang tidak bisa dijangkau dari luar (`internal`).

`client-api` tidak pernah memanggil BMKG atau PVMBG (Problem 2 dan 3).

**Alur event.** Saat Aggregator menyimpan perubahan, ia menulis juga satu baris *outbox* dalam transaksi database yang sama. Proses *relay* lalu mengirim baris itu ke topic Kafka `hazard.events`. Consumer berlangganan ke topic tersebut tanpa diketahui Aggregator (Problem 5). Setiap alur membawa *correlation ID* yang sama dari siklus polling sampai ke consumer.

### Daftar container

Sistem terdiri dari sepuluh container, satu di antaranya hanya tugas sekali jalan. Tabel berikut merangkum port, jaringan, dan penyimpanan masing-masing.

| Container | Port host | Jaringan | Penyimpanan yang dimiliki |
|---|---|---|---|
| `bmkg-mock` | 8081 | app-net | tidak ada (data simulasi di memori) |
| `pvmbg-mock` | 8082 | app-net | tidak ada (data simulasi di memori) |
| `aggregator` | 127.0.0.1:8083 | app, canonical, broker | Canonical Store (tabel `hazards`, `outbox`, `poll_cursor`) |
| `postgres-canonical` | tidak dibuka | canonical-net | volume `canonical-pgdata` |
| `kafka` | tidak dibuka | broker-net | volume `kafka-data` |
| `kafka-init` | – | broker-net | tugas sekali jalan: membuat topic |
| `auth-service` | 8084 | app-net, auth-net | SQLite untuk hash refresh token (volume `auth-data`) |
| `client-api` | 8080 | app-net | tidak ada (tanpa *state*) |
| `consumer-dashboard` | 8091 | broker, app | memori (himpunan `event_id`) |
| `consumer-notifier` | 8092 | broker, app | memori (himpunan `event_id`) |

**Tabel 1.3:** Container, jaringan, dan kepemilikan penyimpanan (dari `docker-compose.yml`).

## 1.4 Teknologi terpilih

Tabel berikut menjawab pertanyaan: teknologi apa yang dipakai untuk tiap komponen, dan kebutuhan sistem mana yang membuatnya dipilih.

| Komponen | Teknologi | Alasan (dikaitkan dengan kebutuhan) |
|---|---|---|
| Semua service | Go 1.25 | Goroutine ringan cocok untuk satu putaran polling per sumber. Image kecil dan build cepat, sehingga membangun ulang satu service tidak lama (Problem 4). Satu bahasa menekan biaya integrasi tiga orang. |
| HTTP dan log | `net/http`, `log/slog` | Tanpa dependensi tambahan. Log JSON membawa `cid` lewat *context*, sesuai syarat log terstruktur. |
| Canonical Store | PostgreSQL 16 dengan kolom JSONB | Kolom tetap untuk field kanonik, `attributes` JSONB untuk field yang bisa berubah: field baru tidak butuh migrasi (Problem 1 dan 4). Transaksi memungkinkan pola outbox (Problem 5). |
| Message broker | Apache Kafka (mode KRaft, satu broker) | Log tahan lama dengan *consumer group* dan *offset*: consumer yang mati dapat menyusul setelah hidup lagi (Problem 5). |
| Klien Kafka | `franz-go` | Go murni, tanpa cgo, sehingga Dockerfile sederhana. |
| Autentikasi | JWT HS256 dan refresh token buram | Token pendek yang diperiksa lokal, jadi jalur baca tidak bergantung pada Auth Service (Problem 2 dan 3). |
| Penyimpanan token | SQLite (`modernc.org/sqlite`) | Hanya hash refresh token yang disimpan; cukup untuk satu instance dan tanpa cgo. |
| Pembatas laju | `golang.org/x/time/rate` | Token bucket: menolak kelebihan beban seketika dengan 429, bukan mengantre (Problem 2). |
| Orkestrasi | Docker Compose, empat jaringan | Satu perintah menyalakan semua. Jaringan *internal* menjadi bukti isolasi penyimpanan. |
| Load test | k6 | Melaporkan throughput, p50/p95/p99, dan laju galat; mendukung ambang lulus. |
| Kualitas | GitHub Actions, `gofmt`, `go vet`, `-race`, gitleaks | Tes otomatis dan pencegahan kredensial ter-commit. |

**Tabel 1.4:** Teknologi per komponen dan alasan pemilihannya.

## 1.5 Asumsi

1. Respons mock berupa JSON array. Parameter `since` bersifat *lebih besar dari* (bukan sama dengan) terhadap `occurred_at`, `issued_at`, atau `reported_at`. Field `issued_at` pada TsunamiWarning adalah tambahan yang diizinkan soal.
2. Warning tsunami dapat terbit *setelah* gempanya. Mock sengaja memberi jeda 2–8 detik. Satu gempa dapat punya beberapa warning dan yang tertinggi menentukan severity.
3. Hanya ada satu instance tiap service pada M1. Konkurensi optimistis (`expectedVersion`) sudah disiapkan untuk M2.
4. Koordinat dan nama gunung api berasal dari tabel referensi statis milik BNPB (`config/volcanoes.json`). Koordinatnya pendekatan untuk simulasi, dan gunung "Prikitiw" fiktif mengikuti cerita soal.
5. Seluruh waktu memakai UTC dan dipotong ke ketelitian mikrodetik.
6. Karantina record yang tidak dapat dipetakan bersifat diagnostik dan disimpan di memori.

7. Endpoint `/admin/*` pada mock (termasuk `extra_fields`, `override`, `/admin/generate`, `/admin/delay`) adalah tambahan untuk demo. Kontrak yang ditetapkan soal tidak diubah.
8. Semua klien hanya membaca. Jalur pelaporan dari lapangan ke BNPB berada di luar M1.
9. Jaringan Docker dianggap tepercaya; tidak ada TLS di dalamnya. Pada produksi, TLS dipasang di ingress.
10. Kredensial pada `.env.example` hanyalah contoh. Nilai sebenarnya diisi di `.env` yang tidak pernah ter-commit.

## 1.6 Petunjuk menjalankan

Ringkasan di sini mengacu ke `README.md` di repositori, yang memuat langkah lengkap, daftar variabel, dan cara memicu setiap skenario.

```
cp .env.example .env            # nilai contoh; .env tidak pernah di-commit
docker compose up --build       # satu perintah untuk seluruh sistem (atau: make up)
make demo-p1                    # lalu demo-p2, demo-p3, demo-isolation, demo-p4, demo-p5, dst.
make reset                      # kembali ke kondisi bersih (hapus container dan volume)
```

Satu-satunya prasyarat adalah Docker dengan Compose v2. Port bawaan: BMKG 8081, PVMBG 8082, Client-Facing API 8080, Auth Service 8084, Aggregator 8083 (hanya *loopback*), consumer 8091 dan 8092. Semua dapat diubah lewat `.env`. Setiap demo memeriksa dirinya sendiri (kode keluar tidak nol bila gagal) dan menyimpan transkripnya ke `docs/evidence/`.

## 1.7 Penggunaan LLM

**Diisi oleh kelompok**

Soal mewajibkan deklarasi penggunaan LLM di laporan beserta lampiran *Form Penggunaan AI.pdf*. Bagian ini sengaja dikosongkan dan diisi sendiri oleh kelompok di akhir:

- alat yang dipakai dan untuk bagian mana (perancangan, kode, laporan) — **[isi]**
- siapa anggota yang memakai dan memahami bagian tersebut — **[isi]**
- lampirkan *Form Penggunaan AI.pdf* — **[lampirkan]**

## 1.8 Istilah penting

Beberapa istilah muncul berulang di laporan ini. Artinya dirangkum di sini agar mudah dicari.

| Istilah | Artinya dalam laporan ini |
|---|---|
| HazardEvent | Format tunggal milik BNPB untuk satu kejadian bahaya, apa pun sumbernya. |
| Canonical Store | Tempat HazardEvent disimpan. Hanya Aggregator yang boleh membukanya langsung. |
| Ringkasan / Mentah | Dua klasifikasi field. Ringkasan boleh dilihat Media; Mentah (koordinat presisi, `attributes`, `source_ref_id`) tidak. |
| Polling | BNPB yang bertanya ke sumber secara berkala. Sumber tidak mengirim sendiri. |
| Outbox | Tabel di database yang menampung event sampai terkirim ke broker, supaya tidak ada event hilang bila broker sedang mati. |
| At-least-once | Setiap event pasti sampai, tetapi bisa sampai lebih dari sekali. |
| Idempoten | Memproses hal yang sama dua kali memberi hasil yang sama dengan sekali. |
| Circuit breaker | Pemutus: setelah sumber gagal berkali-kali, panggilan dihentikan sementara agar tidak memboroskan sumber daya. |
| Token bucket | Pembatas laju: tiap permintaan mengambil satu token; token habis berarti ditolak dengan 429. |
| Stale | Data masih ditampilkan tetapi diberi tanda bahwa sumbernya sedang tidak memperbarui. |
| Correlation ID (`cid`) | Kode yang sama pada semua log satu alur permintaan, dari service pertama sampai terakhir. |

**Tabel 1.5:** Daftar istilah.

# 2 Problem 1 – Arsitektur dan Interoperabilitas Data

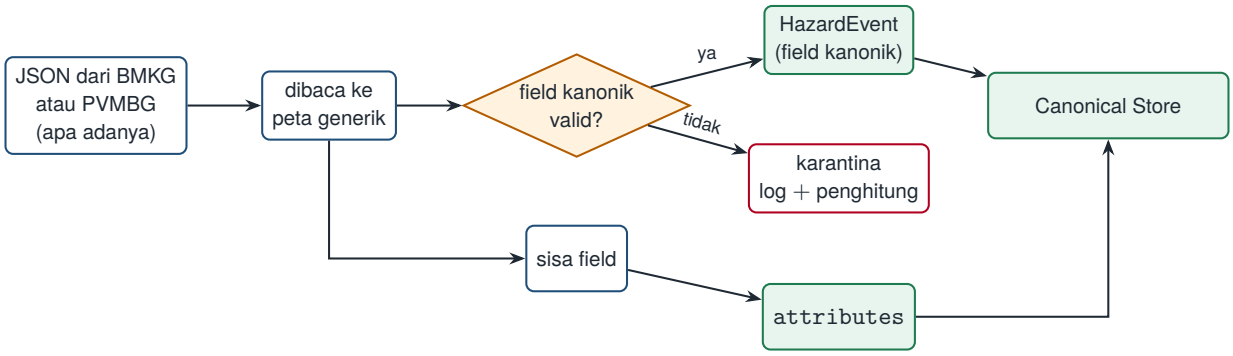
**Singkatnya**

PVMBG bisa menambah field baru kapan saja tanpa memberi tahu siapa pun. Gateway kami **tidak menganggap skema sumber tetap**. Field yang dikenal diperiksa ketat, field lain **diteruskan apa adanya ke `attributes`**, dan hanya perubahan yang mengubah *arti* kontrak (field wajib hilang, tipe berubah, nilai enum baru) yang **dikarantina**, bukan ditebak. Semua ini dibuktikan saat sistem berjalan, tanpa restart.

## 2.1 Rancangan solusi dan mekanisme

**Mekanisme: *tolerant reader* dengan kebijakan *additive-only*.** Aggregator tidak pernah membaca data sumber ke struktur (*struct*) yang tetap. Setiap record dibaca ke peta generik (`map[string]any`), lalu:

1. **Field yang punya padanan kanonik** dibaca secara eksplisit dan divalidasi (harus ada dan bertipe benar), misalnya `magnitude`, `epicenter_lat`, `occurred_at`, `alert_level`, `volcano_id`.
2. **Setiap field lain** diteruskan apa adanya ke `attributes`. Ini mencakup field kontrak yang memang hanya menjadi atribut (`depth_km`, `eruption_count_24h`) dan field yang muncul belakangan (`confidence_level`, atau apa pun).
3. **Record yang melanggar kontrak dengan cara yang tidak didukung** tidak ditebak. Record itu dikarantina: dicatat di log (`mapping_rejected`), dihitung, dan dapat dilihat di `GET /internal/quarantine`. Sisa batch tetap diproses dan penanda kemajuan polling (*watermark*) tetap maju.




**Gambar 2.1:** Alur pemetaan. Sisa field selalu ikut tersimpan; record yang melanggar kontrak berhenti di karantina.

**Kebijakan field tak dikenal (dideklarasikan): diteruskan ke `attributes`.** Kebijakan ini diterapkan di satu tempat saja (fungsi `splitExtras`), sehingga konsisten untuk semua tipe JSON dan kedua sumber. Demo memeriksanya dengan pembanding independen: isi `attributes` harus *persis sama* dengan himpunan field sumber yang tidak punya padanan kanonik, tidak ada yang hilang dan tidak ada yang dikarang.

**Pembaruan record yang sama digabung secara aditif.** Bila record yang sama datang lagi, nilai yang dikirim sumber sekarang menang, tetapi field yang berhenti dikirim tetap tersimpan. Tanpa

aturan ini, mengembalikan skema sumber ke semula akan menimpa record terbaru dengan versi yang lebih miskin dan membuang field yang sudah sempat ditangkap. Bug ini sempat muncul saat pengujian dan kini dijaga oleh `TestSchemaRollbackDoesNotEraseCapturedFields`.

### 2.1.1 Aturan pemetaan

Kontrak soal menetapkan bagaimana tiap field sumber menjadi field HazardEvent. Tabel berikut merangkumnya.

| Kanonik | SeismicEvent (BMKG) | VolcanicReport (PVMBG) |
|---|---|---|
| `hazard_id` | `bmkg-<event_id>` | `pvmbg-<report_id>` |
| `source_ref_id` | `event_id` | `report_id` |
| `hazard_type` | SEISMIC | VOLCANIC |
| `area_name` | `region_name` | nama dari tabel referensi statis BNPB |
| `latitude`, `longitude` | `epicenter_lat`, `epicenter_lon` | dari tabel referensi statis |
| `occurred_at` | `occurred_at` | `reported_at` |
| `severity` | lihat aturan di bawah | `alert_level`: Normal, Waspada, Siaga, Awas menjadi NORMAL, WASPADA, SIAGA, AWAS |
| `attributes` | `magnitude`, `depth_km`, `potential_tsunami`, field TsunamiWarning, dan field tak dikenal | `eruption_count_24h`, `ash_column_height_m`, `confidence_level` bila ada, dan field tak dikenal |

**Tabel 2.1:** Pemetaan ke HazardEvent sesuai kontrak soal.

**Severity seismik.** AWAS bila ada warning dengan `threat_level = Awas`. Selain itu mengikuti `threat_level` warning terkait. Tanpa warning: NORMAL untuk magnitudo di bawah 5,0; WASPADA untuk 5,0 sampai kurang dari 6,5; SIAGA untuk 6,5 ke atas. Bila ada beberapa warning, yang tertinggi menang.

**Warning tsunami datang belakangan.** Warning diambil dari endpoint terpisah dan sering terbit setelah gempanya tersimpan. Aggregator mengaitkannya lewat `related_event_id`, menghitung ulang severity, dan menyimpannya sebagai *versi baru* (`version` naik dan event `updated` diterbitkan). Warning yang gempanya belum terlihat menunggu (dibatasi TTL) dan menahan watermark warning supaya diambil ulang, bukan hilang. Karena `hazard_id` deterministik dan penyimpanan membandingkan isi, polling ulang pada jendela *overlap* tidak menaikkan versi.

### 2.1.2 Jenis perubahan skema yang didukung dan tidak

Karena sumber boleh berubah kapan saja, kami menyatakan dengan tegas perubahan mana yang ditangani otomatis dan mana yang sengaja tidak.

## 2.2 Alasan keputusan dan trade-off

Ada beberapa cara untuk menghadapi skema sumber yang berubah. Tabel berikut membandingkannya dengan pilihan kami.

**Mengapa ini.** Sumber bersifat otonom dan tidak berkoordinasi, jadi mekanisme harus bekerja tanpa kerja sama mereka. Perubahan aditif adalah jenis perubahan yang paling umum dan paling aman ditangani otomatis. Perubahan yang mengubah arti kontrak (ganti nama, ganti tipe,

| Perubahan pada sumber | Perilaku Aggregator | Didukung? |
|---|---|---|
| Field baru bertipe apa pun (angka, teks, objek bersarang, list, `null`) | masuk `attributes`; HazardEvent tetap valid | Ya |
| Field baru pada TsunamiWarning | masuk `attributes` dengan awalan `warning_` agar tidak bertabrakan nama | Ya |
| Field yang hanya berstatus atribut menghilang | atribut itu sekadar tidak ada | Ya |
| Field baru lalu hilang lagi (skema dikembalikan) | record lama mempertahankan nilainya; record baru tidak memilikinya | Ya |
| `volcano_id` tidak ada di tabel referensi | `area_name` berisi id mentah, koordinat `null`, ada peringatan di log | Ya (turun mutu terkendali) |
| Field wajib hilang atau berganti nama | karantina | **Tidak, sengaja** |
| Tipe field wajib berubah (misal `magnitude` jadi teks) | karantina, tidak dipaksa | **Tidak, sengaja** |
| Nilai enum baru (misal `alert_level = Darurat`) | karantina | **Tidak, sengaja** |
| Perubahan arti atau satuan (meter jadi kaki) | tidak terdeteksi | Tidak, di luar jangkauan |
| Bentuk endpoint berubah (bukan array JSON, path berubah) | dianggap kegagalan sumber (circuit breaker) | Tidak |

**Tabel 2.2:** Jenis perubahan skema sumber dan perilakunya.

| Alternatif | Pertimbangan |
|---|---|
| Struct tetap (prototipe Astolfo) | Sederhana dan bertipe, tetapi field baru hilang diam-diam, atau pesan gagal dibaca seluruhnya. Persis masalah di soal. |
| Schema registry | Validasi terpusat. Tetapi menuntut sumber ikut mendaftarkan skema, sedangkan PVMBG mengubah skema *tanpa memberi tahu siapa pun*. Registry tidak akan pernah mutakhir tepat waktu. |
| Protobuf atau Avro (schema evolution) | Evolusi terformalisasi, tetapi sumber mengirim JSON biasa. Lapisan konversi tetap harus tahu skema baru. |
| Pemetaan dinamis lewat berkas konfigurasi | Tanpa redeploy kode, tetapi seseorang tetap harus mengubah konfigurasi saat field baru muncul. Itu masih intervensi manual. |
| **Tolerant reader + additive-only (dipilih)** | Bekerja *sepihak* dari sisi BNPB: tanpa koordinasi, tanpa redeploy, perilakunya deterministik dan mudah dibuktikan. |

**Tabel 2.3:** Alternatif mekanisme untuk evolusi skema.

enum baru) sengaja dihentikan di pintu: menebaknya berisiko salah menafsirkan data kebencanaan. Di sisi penyimpanan, `attributes` adalah dokumen JSONB, sehingga kebijakan ini tidak berubah menjadi migrasi tabel (Problem 4).

## 2.3 Batasan yang disadari

- **Tipe nilai di `attributes` tidak divalidasi.** Ini disengaja (tanpa skema). Perubahan tipe pada field atribut lolos apa adanya.
- **Field tak dikenal ikut diteruskan**, termasuk yang mungkin sensitif. Ini aman karena seluruh `attributes` berklasifikasi Mentah dan Media tidak pernah menerimanya (ditegakkan Client-Facing API, Problem 3).
- **Field yang dihapus sumber tetap menempel pada record lama** akibat penggabungan aditif. Hanya record itu; record baru tidak memilikinya. Ini harga agar informasi tidak pernah dibuang.
- **Ganti nama tidak dikenali sebagai ganti nama:** field lama hilang (karantina bila wajib) dan field baru muncul sebagai atribut.
- **Perubahan arti dan satuan tidak terdeteksi.** Tidak ada pemantauan drift selain log dan penghitung karantina.
- **Karantina hanya di memori** (200 entri terakhir, penghitung sejak start). Ia alat diagnosis, bukan penyimpanan resmi, dan belum ada fitur memutar ulang record yang dikarantina setelah kode diperbaiki.
- Record yang ditolak diingat 15 menit agar polling ulang pada jendela *overlap* tidak mencatatnya berulang.
- Pencocokan enum tidak peka huruf besar-kecil (`SIAGA` sama dengan `Siaga`); nilai lain ditolak.

## 2.4 Bukti penyelesaian

Bukti berasal dari `make demo-p1` (transkrip: `docs/evidence/p1/latest.log`). Demo memeriksa dirinya sendiri dan berakhir dengan `P1 PASS`.

**Cuplikan log: skema berubah saat berjalan (langkah 4 dan 5)**

```
$ POST http://localhost:8082/admin/schema-version {confidence_level:true, extra_fields:{
    lava_dome_volume_m3:125000}}
{"schema_version":2,"confidence_level":true,"extra_fields":{"lava_dome_volume_m3":125000}}
...
{"hazard_id":"pvmbg-VR-58ae09-0029","severity":"AWAS","area_name":"Gunung Lewotobi Laki-laki",
 "attributes":{"ash_column_height_m":5345.9,"confidence_level":0.78,"eruption_count_24h":18,
               "lava_dome_volume_m3":125000}}
   PASS bmkg-mock, pvmbg-mock and aggregator were NOT restarted (StartedAt unchanged)
   PASS all 28 pre-change volcanic hazards are unchanged (same version, same attributes)
   side by side in the store: 5 hazards WITH confidence_level, 28 WITHOUT - both readable
```

**Cuplikan log: perubahan yang tidak didukung dikarantina (langkah 7)**

```
$ POST http://localhost:8082/admin/generate {override:{alert_level:"Darurat"}}   # nilai enum baru
   PASS the valid report issued right after the bad one was stored (the batch was not blocked)
   PASS report VR-58ae09-0035 with alert_level=Darurat was NOT stored
{"source":"PVMBG","record_id":"VR-58ae09-0035","field":"alert_level",
 "reason":"unknown enum value \"Darurat\"","correlation_id":"5dacbefeb284a2f8"}
```

| Kriteria soal | Bukti |
|---|---|
| Skema awal menghasilkan HazardEvent sesuai aturan, kedua sumber | Langkah 2: pembanding independen membaca data mentah dari mock, menghitung sendiri HazardEvent yang diharapkan, lalu mencocokkannya dengan isi Canonical Store. Hasil: 26 event BMKG dan 27 laporan PVMBG, semua lulus; keempat cabang severity terwakili (AWAS 2, NORMAL 12, SIAGA 3, WASPADA 9). |
| Skema berubah saat berjalan, tanpa restart | Langkah 4 memanggil `POST /admin/schema-version`. Langkah 5: waktu `StartedAt` bmkg-mock, pvmbg-mock, dan aggregator tidak berubah. |
| Tetap valid setelah perubahan, tanpa crash atau intervensi kode | Langkah 5: `confidence_level` dan field baru `lava_dome_volume_m3` muncul di `attributes`; pembanding lulus untuk 32 laporan; kedua sumber tetap up. |
| Record sebelum dan sesudah berdampingan | Langkah 6: 28 hazard vulkanik lama tidak berubah (versi dan atribut sama); di store ada 5 hazard *dengan* dan 28 *tanpa* `confidence_level`, keduanya terbaca. |
| Kebijakan konsisten | Langkah 5 (diteruskan) dan langkah 7 (perubahan tak didukung masuk karantina, batch tidak terblokir); `TestUnknownFieldPolicyIsConsistent`. |
| Log ber-correlation ID dan latensi | Langkah 8: satu `cid` melintasi `outbound_call` (dengan `latency_ms`), `hazard_stored`, `poll_ok`, dan `outbox_published`. |

**Tabel 2.4:** Kriteria Problem 1 dan buktinya.

### Cuplikan log: satu alur, satu correlation ID (langkah 8)

```
correlation id: 5dacbefeb284a2f8
{"msg":"outbound_call","target":"pvmbg","latency_ms":2228.447,"status":200}
{"msg":"mapping_rejected"}
{"msg":"hazard_stored","hazard_id":"pvmbg-VR-58ae09-0036","version":1,"outcome":"created"}
{"msg":"poll_ok","fetched":3,"created":1,"quarantined":1}
{"msg":"event_published","latency_ms":23}
{"msg":"outbox_published","hazard_id":"pvmbg-VR-58ae09-0036","version":1,"event_id":"pvmbg-VR-58
    ae09-0036:1"}
```

### Tes otomatis terkait:

- `mapping_test.go`: tabel severity pada batas 4,99 / 5,0 / 6,49 / 6,5, warning terlambat, idempotensi, dan *round-trip* JSON.
- `poller_test.go`: `TestPVMBGSchemaChangeWithoutRestart`, karantina tidak menahan watermark, deduplikasi.
- Tes server mock PVMBG (skema awal dan baru) serta tes kontrak JSON Schema di ketiga service.

## 2.5 Jawaban butir “Di laporan”

**Jawaban butir “Di laporan”**

**Mekanisme yang dipilih:** *tolerant reader* dengan kebijakan *additive-only*. JSON dibaca generik, field kanonik divalidasi eksplisit, sisanya diteruskan ke `attributes`. Schema registry dan Protobuf/Avro tidak dipilih karena sumber tidak berkoordinasi (alasan di bagian 2.2).
**Kebijakan field tak dikenal:** diteruskan ke `attributes` (tidak diabaikan).
**Didukung:** penambahan field bertipe apa pun; hilangnya field yang hanya berstatus atribut; `volcano_id` baru (turun mutu terkendali).

**Sengaja tidak didukung** (masuk karantina, batch tetap berlanjut): penghapusan atau penggantian nama field wajib, perubahan tipe field wajib, dan nilai enum baru. Perubahan arti atau satuan tidak terdeteksi.

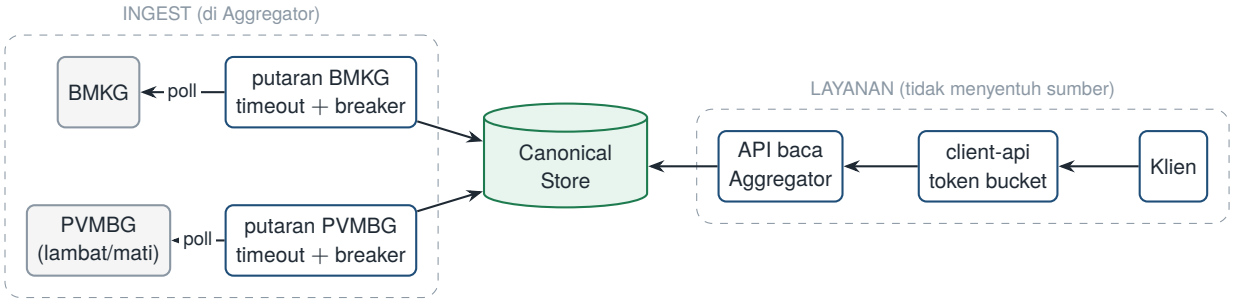
# 3 Problem 2 – Concurrency, Availability, dan Graceful Degradation

**Singkatnya**

Gateway tidak memanggil BMKG atau PVMBG saat melayani klien. Polling dijalankan di **putaran terpisah per sumber** dan hasilnya disimpan di Canonical Store. Klien hanya membaca dari store, jadi PVMBG yang lambat atau mati tidak bisa menahan permintaan lain. Lonjakan beban ditolak seketika dengan HTTP 429, dan saat PVMBG mati data vulkanik terakhir tetap tersaji dengan penanda `stale`.

## 3.1 Rancangan solusi dan mekanisme

**Pokok idenya: pisahkan *mengambil data* dari *melayani klien*.** Prototipe Astolfo memanggil BMKG lalu PVMBG secara berurutan di dalam satu permintaan klien, sehingga lambatnya PVMBG dirasakan semua orang. Pada rancangan kami, jalur layanan tidak pernah menyentuh sumber.




**Gambar 3.1:** Model konkurensi. Dua putaran polling menulis ke store; jalur layanan hanya membaca dari store.

1. **Satu putaran polling per sumber** (`poller/loop.go`). BMKG dan PVMBG masing-masing punya goroutine sendiri dengan timeout permintaan (`SOURCE_TIMEOUT_SEC`), timeout siklus, dan *circuit breaker* (`resilience/breaker.go`). Breaker membuka setelah beberapa kegagalan beruntun, lalu mengirim satu percobaan (*half-open*) setelah `BREAKER_OPEN_SEC`. Siklus berikutnya baru mulai `POLL_INTERVAL_SEC` setelah siklus sebelumnya selesai, jadi sumber yang lambat tidak pernah menerima permintaan bertumpuk.
2. **Jalur layanan tidak memanggil BMKG atau PVMBG sama sekali.** Client-Facing API membaca `GET /internal/*` milik Aggregator, yang membaca Canonical Store. Setiap permintaan klien dilayani goroutine `net/http`-nya sendiri, sehingga permintaan data BMKG tidak berbagi antrean atau kunci dengan apa pun yang menunggu PVMBG.
3. **Pembatas laju (token bucket)** di Client-Facing API (`CLIENT_RATE_LIMIT_RPS` dan `CLIENT_RATE_LIMIT_BURST`, bawaan 500 dan 1000). Kelebihan beban *ditolak seketika* dengan 429 dan `Retry-After`, tidak diantre. `/health` dan `/ready` dikecualikan supaya pemeriksaan kesehatan tetap menjawab saat beban tinggi.
4. **Degradasi terkendali.** Aggregator menyimpan `freshness` untuk tiap sumber (`state`, `last_success_at`, `stale_since`). Saat PVMBG mati, data vulkanik terakhir tetap disajikan dari store. Client-Facing API menandai tiap item `stale: true` dan meneruskan

`freshness.PVMBG.stale_since`. Tidak ada jalur yang mengubah matinya satu sumber menjadi galat 500 atau permintaan yang menggantung.

### 3.1.1 Bagaimana model ini mencegah tiga kegagalan

| Kegagalan di soal | Mekanisme yang mencegahnya |
|---|---|
| Head-of-line blocking | Tidak ada rantai panggilan berurutan BMKG lalu PVMBG. Putaran polling terpisah per sumber, dan permintaan klien hanya menyentuh store. PVMBG yang butuh 3 detik tidak berada di jalur permintaan BMKG-only mana pun. |
| Resource exhaustion | Token bucket menolak kelebihan beban *sebelum* menyentuh Aggregator. Setiap panggilan keluar punya timeout. Breaker membuat sumber yang mati tidak menghabiskan goroutine dan koneksi: polling dilewati, bukan ditumpuk. Batasnya dipasang pada *laju* permintaan, bukan pada jumlah permintaan yang sedang berjalan. Karena permintaan dilayani dari store lokal dalam milidetik (p99 sekitar 28 ms pada load test), laju yang dibatasi ikut membatasi jumlah permintaan yang berjalan bersamaan. |
| Tanpa degradasi terkendali | Data terakhir dari Canonical Store disajikan bersama `stale` dan `stale_since`. Sumber lain tetap segar. |

**Tabel 3.1:** Pemetaan tiga kegagalan ke mekanisme pencegahnya.

## 3.2 Alasan keputusan dan trade-off

Model konkurensi di atas bukan satu-satunya pilihan. Tabel berikut menunjukkan alternatif yang kami pertimbangkan.

| Alternatif | Mengapa tidak dipilih |
|---|---|
| Memanggil BMKG dan PVMBG *paralel* di tiap permintaan klien | Menghilangkan pemblokiran berurutan, tetapi permintaan tetap menunggu PVMBG yang paling lambat (3 detik) atau harus timeout. Tiap lonjakan klien juga melipatgandakan beban ke sumber yang sudah lemah. |
| Cache dengan TTL di Client-Facing API | Menyembunyikan masalah, bukan menyelesaikannya: data basi tanpa penanda, dan ada dua tempat menyimpan satu fakta (melanggar "satu storage satu pemilik"). |
| Antrean dan *worker pool* untuk permintaan klien | Membatasi konkurensi, tetapi menukar penolakan cepat dengan latensi yang membengkak karena antrean mengisi memori. |
| Semaphore konkurensi (batas permintaan sedang berjalan) | Melindungi dari permintaan lambat. Di sini permintaan cepat (store lokal), sehingga laju adalah ukuran yang lebih tepat. |
| **Putaran per sumber, store sebagai penyangga, token bucket (dipilih)** | Terpisah secara struktural. Perilaku saat gagal deterministik dan dapat didemokan. |

**Tabel 3.2:** Alternatif model konkurensi.

**Harga yang diterima.** Data yang dilayani *selalu* berasal dari store, jadi kesegarannya setara interval polling (3 detik). Selain itu, kebijakan laju bersifat global (lihat bagian berikut).

## 3.3 Batasan yang disadari

- **Pembatas laju bersifat global**, bukan per klien, dan dipasang *sebelum* autentikasi. Satu klien agresif atau banjir permintaan tanpa token dapat menghabiskan jatah klien lain. Ini cukup untuk membuktikan perlindungan sumber daya pada M1; pembatasan per klien adalah per-

baikan untuk M2.

- **Tidak ada batas jumlah koneksi bersamaan** selain laju. Beban koneksi lambat (*slowloris*) hanya dibatasi `ReadHeaderTimeout` 5 detik.
- **Satu instance** Client-Facing API dan Aggregator. Skala horizontal (termasuk pembatas laju terdistribusi) masuk M2.
- Penanda `stale` pada permintaan satu hazard membutuhkan satu panggilan tambahan ke `/internal/sources/status`.
- Data saat outage adalah potret terakhir; tidak ada estimasi atau ekstrapolasi.
- Hasil load test berasal dari **satu mesin** (k6, Docker, dan semua service berbagi CPU mesin Windows 11). Angka absolutnya bukan klaim kapasitas produksi. Throughput sustained dibatasi *think time* 0,1 detik per VU (sekitar 450 permintaan per detik untuk 50 VU), bukan oleh server.

## 3.4 Konfigurasi dan tooling load test

Load test dijalankan dengan k6. Berikut konfigurasi yang dipakai agar hasilnya dapat diulang.

| Item | Nilai |
|---|---|
| Tool | **k6** (biner lokal, atau `grafana/k6:0.54.0` lewat Docker bila tidak ada) |
| Skrip | `scripts/loadtest/p2.js`, dijalankan oleh `scripts/loadtest/run-p2.sh` (bagian dari `make demo-p2`, langkah E dan F) |
| Target | `GET /v1/hazards?hazard_type=SEISMIC&limit=20` pada Client-Facing API port 8080, peran Lapangan |
| Uji *sustained* | **50 VU konstan selama 60 detik**, think time 0,1 detik (`K6_VUS`, `K6_DURATION`, `K6_THINK_S`) |
| Uji *overload* | 150 VU selama 20 detik tanpa think time, sengaja melampaui batas laju 500 permintaan per detik |
| Autentikasi | Tiap VU login sendiri lalu memperbarui token lewat `POST /v1/refresh` 10 detik sebelum `exp` (TTL 60 detik). Versi awal memakai satu token untuk 60 detik sehingga mengukur kedaluwarsanya sendiri sebagai galat; ini sudah diperbaiki |
| Klasifikasi respons | **200** = sukses; **429** = penolakan terkendali (dilaporkan, bukan galat); **tak tertangani** = selain keduanya (5xx, 401/403, timeout) |
| Ambang lulus | `unhandled < 1%` dan `p95 < 300 ms`. `make demo-p2` gagal bila terlampaui |
| Kondisi sistem | PVMBG diberi delay 3 detik pada langkah A; seluruh stack dari `make reset` |

**Tabel 3.3:** Konfigurasi load test.

## 3.5 Bukti penyelesaian

Angka berasal dari `make demo-p2` dan `make demo-p2-aggregator`. Berkas mentahnya ada di `docs/evidence/p2/`. Angka berbeda dari run ke run; yang ditegakkan skrip adalah kriteria lulus-gagalnya. Contohnya, pada dua run berbeda p95 BMKG-only bernilai 62,9 ms dan 152,4 ms, dan keduanya jauh di bawah batas 300 ms. Angka di tabel berasal dari run terbaru (8 Oktober 2026, setelah `make reset`).

**Catatan tentang overload.** Dalam 20 detik, token bucket dengan 500 token per detik dan *burst* 1000 secara teori meloloskan sekitar $500 \times 20 + 1000 \approx 11.000$ permintaan. Hasil ukur 10.989 permintaan sukses sangat dekat dengan angka itu, artinya pembatas bekerja sesuai konfigurasi,

| Kriteria soal | Hasil | Berkas bukti |
|---|---|---|
| **p95 BMKG-only di bawah 300 ms** saat PVMBG delay 3 detik, bersamaan dengan permintaan data PVMBG | 4 pembaca data vulkanik paralel + 40 permintaan BMKG-only: **p95 152,4 ms**, 0 galat. Pembaca vulkanik p95 154–190 ms. | `latest.log`, langkah A |
| **Load test** ≥50 koneksi, ≥60 detik | 26.752 permintaan, **445,1 req/s**, **p50 9,4 ms, p95 19,6 ms, p99 28,3 ms**, maksimum 69 ms. Status 200: 26.752; 429: 0; lainnya: 0. **Galat tak tertangani 0%** (batas 1%). 50 perpanjangan token berjalan di tengah uji. | `k6-summary.json` |
| **Overload terkendali** (150 VU) | 81.112 permintaan (sekitar 4.050 req/s): 200: 10.989; **429: 70.123 (86,5%)**; **lainnya: 0**; p95 68 ms. `/health` tetap 200 sesudahnya; tidak ada crash. | `k6-overload-summary.json`, `latest.log`, langkah F |
| **Outage PVMBG** (mode `error`) | 27 item vulkanik tetap tersaji, **semuanya `stale: true`**, `freshness.PVMBG.stale_since` terisi; pembacaan vulkanik p95 19,7 ms, 0 galat; event BMKG baru tetap masuk. | `latest.log`, langkah B; `aggregator-latest.log` |
| Outage mode `hang` (tidak menjawab) | Pembacaan seismik p95 12,1 ms; klien tidak menggantung. Di sisi Aggregator, p95 9,0 ms. | `latest.log`, langkah C |
| **Pulih tanpa restart** | PVMBG kembali up, laporan baru ter-ingest, `StartedAt` lima container tidak berubah. | `latest.log`, langkah D |
| Correlation ID lintas service | Satu `cid` melintasi `request` client-api, `outbound_call`, dan `request` aggregator. | `cid-trace.log` |

**Tabel 3.4:** Kriteria Problem 2 dan buktinya.

dan semua sisanya ditolak dengan 429 tanpa satu pun 5xx.

### Sisi Aggregator saat PVMBG mati (`make demo-p2-aggregator`)

```
A. PVMBG answers in 3 s:   Aggregator API seismic p95 = 19.3 ms, non-200 = 0
   PASS a new BMKG event was ingested 2 s after it appeared, although PVMBG is slow
B. PVMBG outage (error):   PASS stale_since is set: 2026-10-07T19:44:08.985391266Z
   PASS all 87 volcanic hazards are still served from the Canonical Store (now 88)
   PASS volcanic reads during the outage: p95 77.5 ms, 0 errors (no 500, no hanging request)
   PASS while the circuit is open polls are skipped, not sent to the dead service (
     skipped_by_breaker=44)
C. PVMBG outage (hang):    PASS Aggregator API answers in p95 9.0 ms
   last_error: ... net/http: request canceled (Client.Timeout exceeded while awaiting headers)
D. Outage ends:            PASS PVMBG is 'up' again, stale_since=null
   PASS nothing was restarted (StartedAt of bmkg-mock, pvmbg-mock, aggregator unchanged)
```

### Satu correlation ID melintasi dua service (`cid-trace.log`)


```
{"msg":"request","service":"aggregator","path":"/internal/hazards","status":200,"duration_ms
    ":0.885,"cid":"demo-p2-trace-0001"}
{"msg":"outbound_call","service":"client-api","target":"aggregator","latency_ms":1.463,"status
    ":200,"cid":"demo-p2-trace-0001"}
{"msg":"request","service":"client-api","path":"/v1/hazards","status":200,"duration_ms":2.107,"cid
    ":"demo-p2-trace-0001"}
```


**Tes otomatis terkait:**

- `TestSlowSourceDoesNotBlockFastSourceAndCyclesNeverOverlap`: sumber lambat tidak menahan sumber cepat.

- `TestStaleMarkerWhenSourceIsDown`: penanda `stale` untuk semua peran.
- `TestUpstreamDownIs503NotHang`: Aggregator tak terjangkau dijawab cepat, tidak menggantung.
- `TestShedsExcessWith429ButKeepsHealthAlive`: kelebihan beban ditolak 429, `/health` tetap hidup.
- `resilience_test.go`: perilaku circuit breaker.

## 3.6 Jawaban butir "Di laporan"

**Jawaban butir "Di laporan"**

**Model konkurensi.** Putaran polling terpisah per sumber (dengan timeout dan circuit breaker) yang menulis ke Canonical Store. Jalur layanan hanya membaca store lewat API Aggregator, dengan token bucket di depannya. Pemetaan ke tiga kegagalan ada di bagian 3.1.1.
**Konfigurasi dan tooling load test.** k6; 50 VU selama 60 detik (sustained) dan 150 VU selama 20 detik (overload); respons diklasifikasi 200/429/tak tertangani; ambang `unhandled < 1%` dan `p95 < 300 ms`. Rinciannya di bagian 3.4.

# 4 Problem 3 – Autentikasi dan Kepercayaan antar-Service

**Singkatnya**

Prinsipnya: **setiap domain kepercayaan punya kredensial sendiri, diterbitkan dan diperiksa oleh pemiliknya, dan tidak ada satu rahasia pun yang dipakai di dua domain**. Kunci BMKG tidak membuka PVMBG, kunci PVMBG tidak membuka BMKG, dan token klien tidak membuka keduanya. Untuk klien, Auth Service menerbitkan token JWT berumur 60 detik per peran, dan Client-Facing API menegakkan hak akses *di server*: Media tidak pernah menerima field Mentah.

## 4.1 Rancangan solusi dan mekanisme

Ada lima domain kepercayaan. Masing-masing dibuktikan terpisah di bagian bukti.

| Domain | Kredensial | Diperiksa oleh | Jika bocor |
|---|---|---|---|
| BMKG | `X-BMKG-Key`, kunci buram | bmkg-mock | hanya BMKG |
| PVMBG | `Authorization: Bearer pvmbg.v1.<klaim>.<HMAC>`, struktur berbeda | pvmbg-mock | hanya PVMBG |
| API internal Aggregator | `X-Service-Key` | Aggregator | jalur internal; tidak membuka BMKG, PVMBG, atau klien |
| Operator mock | `X-Admin-Token`, satu per mock | masing-masing mock | `/admin/*` mock itu saja |
| Klien downstream | `client_id` dan `client_secret` per peran, lalu JWT | Auth Service (penerbit) dan Client-Facing API (pemeriksa) | hanya peran klien itu |

**Tabel 4.1:** Lima domain kepercayaan dan kredensialnya masing-masing.

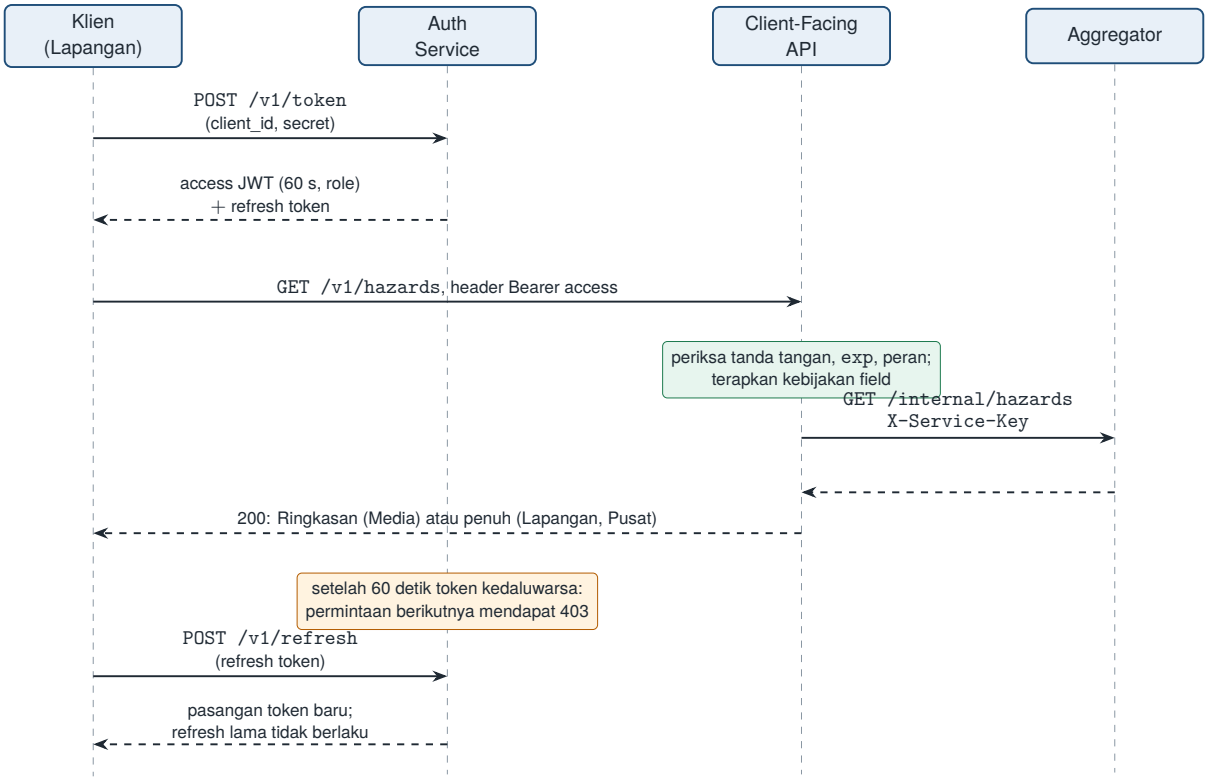
Aggregator memegang *token* PVMBG tetapi **bukan rahasia penanda tangannya**. Jadi ia bisa memanggil PVMBG, tetapi tidak bisa mencetak token baru atau mengadministrasi PVMBG.

### 4.1.1 Klien downstream: Media, Tim Lapangan, BNPB Pusat

- **Tiga identitas yang benar-benar berbeda** (`AUTH_MEDIA_*`, `AUTH_LAPANGAN_*`, `AUTH_PUSAT_*`, masing-masing dengan id dan secret sendiri), bukan satu akun dengan bendera peran di sisi klien. Peran ditentukan Auth Service dari identitas yang lolos login, lalu ditulis ke klaim `role` *di dalam token bertanda tangan*. Klien tidak bisa memilih perannya.
- **Access token:** JWT HS256 dengan klaim `sub`, `role`, `jti`, `iat`, `exp`. TTL **60 detik** (`AUTH_ACCESS_TTL_SEC`, dapat diubah).
- **Refresh token:** 256 bit acak, **tidak pernah disimpan polos**; yang disimpan hanya hash SHA-256 di SQLite milik Auth Service (volume yang hanya dipasang pada container itu). Sekali pa-

kai: `POST /v1/refresh` menandai token lama `revoked` dalam transaksi yang sama dengan pembacaannya, lalu menerbitkan pasangan baru. TTL `AUTH_REFRESH_TTL_SEC` (24 jam).

- **Penegakan di sisi BNPB, pada setiap permintaan** (`services/client-api`):
  1. Tanpa `Authorization: Bearer` → **401**. Token rusak, kedaluwarsa, salah tanda tangan, atau peran tak dikenal → **403**.
  2. **Media** menerima *hanya* field Ringkasan lewat *whitelist* (`policy.FilterItem`). Field baru yang kelak muncul di HazardEvent tidak otomatis bocor ke Media.
  3. Permintaan Media atas field Mentah (`fields=latitude`, `include=mentah`, dan sejenisnya) **ditolak 403**. Aturannya juga whitelist (`policy.CheckQuery`): setiap nilai `fields` harus field Ringkasan, dan `include` hanya boleh kosong atau `summary`. Variasi penulisan (huruf besar, spasi, `fields[]`, parameter berulang, daftar dipisah koma, nama yang tak dikenal) tidak lolos.
  4. `/v1/sources/status` hanya untuk Lapangan dan Pusat; Media mendapat 403.
- Kredensial, kunci penanda tangan, dan konfigurasi dibaca dari *environment*. Service berhenti dengan pesan jelas bila variabelnya kosong (`obs.MustEnv`). Repositori hanya memuat `.env.example` berisi nilai contoh.




**Gambar 4.1:** Alur autentikasi klien: login, akses, kedaluwarsa, lalu *refresh* tanpa login ulang.

## 4.2 Alasan keputusan dan trade-off

Tabel berikut membandingkan beberapa cara mengamankan akses klien dengan pilihan kami.

**Harga yang diterima.** `AUTH_JWT_SECRET` dipegang *dua* service milik BNPB: Auth Service dan Client-Facing API. Rahasia ini independen dari kredensial BMKG, PVMBG, kunci layanan, dan token admin, sehingga kebocorannya tidak merambat ke domain lain. Konsekuensinya, bila Client-Facing API dibobol, penyerang dapat mencetak token klien. Itulah alasan pilihan asime-

| Alternatif | Mengapa tidak dipilih |
|---|---|
| Satu kunci untuk semua (kondisi awal Astolfo) | Persis masalah di soal: kunci Media bocor berarti data mentah bocor, dan kunci BMKG menembus PVMBG. |
| API key statis per klien, tanpa token | Tidak ada kedaluwarsa dan refresh, padahal Lapangan wajib bertoken pendek. Kunci yang bocor berlaku selamanya. |
| Token buram dengan introspeksi ke Auth Service tiap permintaan | Pencabutan seketika, tetapi setiap permintaan klien menambah satu panggilan jaringan dan Auth Service menjadi titik gagal bagi jalur baca. Ini bertentangan dengan tujuan Problem 2. |
| JWT asimetris (RS256 atau EdDSA) | Lebih aman: Client-Facing API cukup memegang kunci publik, sehingga bila API dibobol, penyerang tidak bisa mencetak token. Tetapi menambah pengelolaan kunci. **Dicatat sebagai perbaikan M2.** |
| OAuth2 atau Keycloak penuh | Berlebihan untuk tiga klien read-only, dan sulit didemokan dengan TTL 60 detik. |
| **JWT HS256 dengan refresh token buram (dipilih)** | Verifikasi lokal (jalur baca tidak bergantung pada Auth Service), TTL pendek, refresh dapat dicabut per token. Sederhana dan mudah dibuktikan. |

**Tabel 4.2:** Alternatif mekanisme autentikasi klien.

tris dicatat sebagai langkah berikutnya.

## 4.3 Batasan yang disadari

- **Access token lama tidak dicabut seketika saat refresh.** JWT bersifat *stateless*, jadi token yang belum kedaluwarsa tetap berlaku sampai `exp` (paling lama 60 detik). Yang dicabut seketika adalah *refresh token* lama (terbukti: dipakai ulang mendapat 403). Pencabutan seketika menuntut introspeksi atau daftar `jti` yang selalu diperbarui.
- **Tidak ada deteksi pemakaian ulang refresh token.** Keluarga token tidak dicabut ketika refresh yang sudah terpakai muncul lagi.
- **`/v1/token` tidak dibatasi percobaan** (tanpa lockout atau pembatasan laju per klien). Perbandingan secret memakai waktu konstan.
- **Pembatas laju Client-Facing API bersifat global** dan dipasang sebelum autentikasi, jadi banjir permintaan tanpa token ikut menghabiskan jatah. Per klien setelah autentikasi adalah perbaikan M2.
- Token kedaluwarsa dijawab **403** (bukan 401), mengikuti kontrak yang dibekukan di `docs/contracts.md`.
- Tidak ada TLS di dalam jaringan Compose; secret klien statis di *environment* tanpa perkakas rotasi; SQLite Auth Service hanya cocok untuk satu instance (M2 butuh penyimpanan bersama).

## 4.4 Bukti penyelesaian

Setiap kriteria di soal dibuktikan oleh demo yang memeriksa dirinya sendiri atau oleh tes otomatis.

### Kredensial saling menolak (`make demo-mock-auth`)

```
[1] BMKG mock (X-BMKG-Key)
   PASS own key                          -> HTTP 200
   PASS no credential                    -> HTTP 401
```

| Kriteria soal | Bukti |
|---|---|
| Kredensial BMKG ditolak PVMBG dan sebaliknya (401/403) | `make demo-mock-auth`: kunci BMKG sebagai Bearer → 403; token PVMBG di `X-BMKG-Key` → 403; token dirusak → 403; tanpa kredensial → 401; token admin silang → 403. Setiap container hanya memegang rahasianya sendiri. |
| Media hanya menerima Ringkasan; permintaan field mentah ditolak server | `make demo-p3` langkah B (isi respons tanpa `latitude`, `longitude`, `attributes`, `source_ref_id`) dan langkah C (sepuluh bentuk permintaan Mentah → 403; permintaan hanya-Ringkasan → 200). Tes: `TestMediaNeverReceivesMentahOnListOrGet`, `TestMediaExplicitMentahRequestIs403`. |
| Token Lapangan kedaluwarsa alami, refresh tanpa login, token lama ditolak | `make demo-p3` langkah F: token valid → tunggu sampai `exp` → 403 → `POST /v1/refresh` → 200 dengan token baru *tanpa* `client_secret` → refresh lama dipakai ulang → 403. |
| Tidak ada kredensial ter-hardcode atau ter-commit; `.env.example` ada | `make secrets`: `.env` tidak pernah ter-commit, gitleaks bersih (riwayat dan *working tree*), tidak ada literal kredensial di kode Go non-test, dan `.env.example` memuat seluruh variabel. |
| 401 dibedakan dari 403 | `make demo-p3` langkah A; `TestMissingTokenIs401AndBadTokenIs403`: tanpa token 401; sampah, kedaluwarsa, kunci lain, tanpa `exp`, peran tak dikenal, dan `alg=none` semuanya 403. |

**Tabel 4.3:** Kriteria Problem 3 dan buktinya.

```
   PASS wrong key                       -> HTTP 403
   PASS PVMBG bearer token              -> HTTP 403
[2] PVMBG mock (Authorization: Bearer <pvmbg-token>)
   PASS own token                       -> HTTP 200
   PASS BMKG key presented as bearer    -> HTTP 403
   PASS tampered PVMBG token            -> HTTP 403
   claims: {"aud":"pvmbg-api","exp":...,"iss":"pvmbg-geologi","sub":"bnpb-aggregator"}
[5] Each container only holds the secrets of its own trust domain
   PASS bmkg-mock has no PVMBG/internal secret; pvmbg-mock has no BMKG/internal secret;
        the aggregator holds tokens but cannot mint or administer
```

### Media, Mentah, dan token kedaluwarsa (`make demo-p3`)

```
[1] PASS no header -> 401        PASS empty Bearer -> 401        PASS garbage JWT -> 403
[2] PASS media list omits all Mentah fields on first item
[3] PASS media ?fields=latitude -> 403         PASS media ?include=mentah -> 403
    PASS media ?fields=Latitude -> 403         PASS media ?fields%5B%5D=longitude -> 403
    PASS media ?include=raw -> 403             PASS media asking only for Ringkasan fields -> 200
[4] PASS media /v1/sources/status -> 403       PASS lapangan /v1/sources/status -> 200
[6] expires_in=60s (configured AUTH_ACCESS_TTL_SEC=60)
    PASS fresh access token works
    PASS expired access token -> 403
    PASS refreshed access works without client_id/secret again
    PASS rotated refresh token cannot be reused (403)
```

## 4.5 Jawaban butir "Di laporan"

**Mekanisme yang dipakai**

Dua lapis. **(1) Antar-instansi:** kredensial BMKG (kunci buram) dan PVMBG (token bertanda tangan HMAC dengan struktur lain; rahasia penanda tangan hanya di mock PVMBG) saling asing dan masing-masing diperiksa pemiliknya. **(2) Klien downstream:** Auth Service menerbitkan JWT HS256 berumur 60 detik per peran ditambah refresh token buram sekali pakai. Client-Facing API memeriksa setiap permintaan dan menegakkan klasifikasi Ringkasan/Mentah di server.

**Mengapa tidak membuka celah baru**

- **Pemalsuan token:** algoritma dikunci ke HS256 (`alg=none` dan algoritma lain ditolak; diuji). `exp` wajib ada dan diperiksa (token bertanda tangan tanpa `exp` ditolak; diuji). Peran harus salah satu dari tiga nilai yang dikenal.
- **Eskalasi peran dari sisi klien:** peran hanya berasal dari klaim bertanda tangan. Header atau parameter buatan klien (misalnya `X-Role`) tidak dibaca. Media tidak punya jalur ke field Mentah karena respons dibentuk dengan whitelist, bukan dengan menghapus field yang dikenal.
- **Kebocoran lintas domain:** rahasia tiap domain terpisah dan tidak dipakai ulang. Aggregator tidak memegang rahasia penanda tangan PVMBG (diverifikasi `demo-mock-auth`).
- **Pencurian dari penyimpanan:** refresh token hanya tersimpan sebagai hash. Basis datanya tidak dapat dijangkau service lain. Pencurian refresh token dibatasi oleh sifat sekali pakai dan TTL.
- **Jendela penyalahgunaan:** token curian berguna paling lama 60 detik (access) atau sekali (refresh).
- **Pembatasan yang diakui** ada di bagian 4.3: tidak ada pencabutan access token seketika, dan HS256 dibagi dua service BNPB.

# 5 Problem 4 – Arsitektur Microservice dan Flexible Storage

**Singkatnya**

Sistem dipecah menjadi container yang masing-masing punya satu alasan untuk berubah: mock sumber, Aggregator, Auth Service, Client-Facing API, dan consumer. Canonical Store memakai **PostgreSQL dengan kolom JSONB**, jadi field baru seperti `confidence_level` tidak memerlukan perubahan tabel. Jaringan Docker *internal* membuat database **secara fisik tidak terjangkau** oleh service selain Aggregator.

**Status bukti Problem 4**

Ketiga kriteria terbukti lewat satu skrip, `make demo-p4`: (A) `pvmbg-mock` dihentikan, dibangun ulang, dan dijalankan ulang sementara container lain tetap berjalan; (B–D) field baru masuk ke PostgreSQL JSONB tanpa migrasi dan data lama tetap terbaca; (E) hanya Aggregator yang dapat menjangkau Canonical Store.

## 5.1 Rancangan solusi dan mekanisme

### 5.1.1 Pembagian service

Aturan yang dipakai untuk menarik batas: **satu container, satu alasan untuk berubah, dan satu pemilik untuk setiap data**. Perbaikan di satu alasan tidak boleh memaksa container lain dibangun ulang.

| Container | Tanggung jawab tunggal | Berubah ketika |
|---|---|---|
| `bmkg-mock`, `pvmbg-mock` | Menyimulasikan lembaga lain; bukan bagian BNPB | kontrak sumber berubah (di luar kendali BNPB) |
| `aggregator` | Mengambil data, memetakannya, memiliki Canonical Store, dan menerbitkan event | aturan pemetaan atau ingest berubah |
| `auth-service` | Menerbitkan dan memperbarui token klien | kebijakan autentikasi berubah |
| `client-api` | Menyajikan data, menegakkan hak akses dan batas laju; tanpa *state* | kebijakan field atau batas laju berubah |
| `consumer-*` | Satu kebutuhan pihak yang berlangganan (dashboard, notifikasi) | kebutuhan pihak itu berubah |
| `postgres-canonical`, `kafka` | Infrastruktur penyimpanan dan penyaluran event | versi atau konfigurasi infrastruktur berubah |

**Tabel 5.1:** Batas service dan alasan masing-masing berubah.

Contoh yang dituntut soal: *perbaikan kecil di autentikasi*. Perbaikan itu hanya menyentuh `auth-service` (atau `client-api` untuk pemeriksaannya). Aggregator, mock, dan consumer

tidak ikut dibangun ulang.

### 5.1.2 Aturan kepemilikan penyimpanan

Setiap penyimpanan hanya dimiliki satu service. Komponen lain meminta data lewat API pemiliknya.

| Penyimpanan | Pemilik | Cara komponen lain memperoleh data |
|---|---|---|
| Canonical Store (PostgreSQL) | `aggregator` | API internal Aggregator (`/internal/*`) dengan `X-Service-Key` |
| Hash refresh token (SQLite) | `auth-service` | endpoint `/v1/token` dan `/v1/refresh` |
| Topic Kafka `hazard.events` | `aggregator` sebagai penerbit | berlangganan sebagai *consumer group* |
| Himpunan `event_id` consumer | masing-masing consumer | tidak dibagikan |

**Tabel 5.2:** Kepemilikan penyimpanan.

### 5.1.3 Penyimpanan yang fleksibel: PostgreSQL dengan JSONB

Tabel `hazards` memakai kolom tetap hanya untuk identitas dan field kanonik. Semua yang bisa berubah masuk ke satu kolom `attributes` bertipe JSONB.

```
CREATE TABLE IF NOT EXISTS hazards (
  hazard_id     text PRIMARY KEY,
  source        text NOT NULL,     source_ref_id text NOT NULL,
  hazard_type   text NOT NULL,     severity      text NOT NULL,
  area_name     text NOT NULL,
  latitude      double precision,  longitude     double precision,
  occurred_at   timestamptz NOT NULL,  ingested_at timestamptz NOT NULL,
  updated_at    timestamptz NOT NULL,  version     integer NOT NULL,
  attributes    jsonb NOT NULL DEFAULT '{}'      -- field dinamis: tanpa DDL saat field baru muncul
);
```

Potongan di atas disederhanakan: definisi aslinya (`adapters/postgres/repo.go`) juga memuat indeks dan aturan pengurutan teks untuk `hazard_id`, tetapi tidak ada kolom lain. Perintah `CREATE ... IF NOT EXISTS` berjalan *sekali saat container start*, bukan sebagai langkah migrasi setiap kali sumber berubah. Repositori tidak memuat direktori migrasi maupun berkas SQL terpisah. Ketika PVMBG mulai mengirim `confidence_level`, Aggregator tidak di-restart dan tidak ada DDL yang dijalankan; field itu langsung masuk ke `attributes`.

### 5.1.4 Isolasi jaringan sebagai bukti kepemilikan

Janji "hanya Aggregator yang memegang database" tidak dibiarkan sebagai janji. Compose membuat empat jaringan, tiga di antaranya *internal* (tidak bisa diakses dari luar dan tidak bisa dipakai untuk membuka port ke host):

## 5.2 Alasan keputusan dan trade-off

### 5.2.1 Batas service: mengapa dipecah seperti ini

Ada beberapa cara memotong sistem menjadi service. Tabel berikut membandingkannya dengan pilihan kami.

| Jaringan | Internal? | Anggota |
|---|---|---|
| `app-net` | tidak | mock, aggregator, auth-service, client-api, consumer (hanya port `/health` dan `/stats`) |
| `canonical-net` | ya | **hanya** aggregator dan postgres-canonical |
| `broker-net` | ya | kafka, aggregator (penerbit), consumer |
| `auth-net` | ya | auth-service |

**Tabel 5.3:** Jaringan Docker dan anggotanya.

| Pembagian alternatif | Pertimbangan |
|---|---|
| Monolit modular (satu container) | Paling sederhana, tetapi inilah kondisi yang dikeluhkan soal: perbaikan autentikasi memaksa seluruh sistem dibangun ulang. |
| Dipecah per lapisan teknis (poller, mapper, penyimpan, penerbit) | Terlalu halus. Hazard dan baris outbox harus ditulis dalam *satu transaksi*; bila dipisah, transaksi itu menjadi transaksi terdistribusi. Pemetaan juga membutuhkan hasil polling yang sama, sehingga memisahkannya hanya menambah satu lompatan jaringan tanpa manfaat deploy mandiri. |
| Dipecah per sumber (adapter BMKG dan adapter PVMBG di container sendiri) | Menarik: perubahan skema PVMBG hanya membangun ulang adapter PVMBG. Tidak diambil pada M1 karena setiap adapter butuh kontrak, container, dan jalur ke store tersendiri, sedangkan di dalam Aggregator kedua sumber sudah berjalan di putaran terpisah. **Kandidat untuk M2.** |
| **Dipecah per alasan berubah (dipilih)** | Sumber, Aggregator (pemilik data), autentikasi, penyajian, dan consumer dapat dibangun ulang secara mandiri. Pasangan yang harus konsisten secara transaksi tetap satu container. |

**Tabel 5.4:** Alternatif pembagian service.

**Harga yang diterima.** Mengubah aturan pemetaan berarti membangun ulang Aggregator, termasuk polling-nya. Ini tidak berbahaya karena penanda kemajuan polling (*watermark*) tersimpan di database dan Aggregator tidak bergantung pada urutan start (tanpa `depends_on` ke mock atau store; ia mencoba ulang koneksi sampai 90 detik). Auth Service, Client-Facing API, dan consumer tidak ikut tersentuh.

### 5.2.2 Pilihan penyimpanan

Kebutuhan utamanya: field baru dari sumber tidak boleh memaksa migrasi tabel. Tabel berikut membandingkan pilihan yang masuk akal.

**Konsekuensi pilihan.**

- **Untung:** field baru tanpa migrasi; hazard dan entri outbox ditulis atomik, yang menjadi dasar Problem 5; keyset pagination dengan indeks (`hazard_type, occurred_at, hazard_id`).
- **Rugi:** isi `attributes` tidak divalidasi tipenya dan belum diindeks, sehingga API belum mendukung penyaringan berdasarkan nilai atribut. Bila kelak dibutuhkan, indeks GIN dapat ditambahkan tanpa mengubah kontrak.
- **Keputusan kecil yang disengaja:** kolom `outbox.payload` bertipe `json`, bukan `jsonb`, supaya pesan tersimpan persis seperti dibuat (JSONB menata ulang kunci).

| Alternatif | Kelebihan | Konsekuensi bagi sistem ini |
|---|---|---|
| Relasional dengan kolom tetap | Skema tegas, query kuat | Setiap field baru butuh migrasi dan deploy bersama: masalah Problem 1 kembali lewat jalur storage. **Ditolak.** |
| Tabel atribut terpisah (EAV) | Fleksibel tanpa DDL | Query menjadi rumit dan lambat (banyak *join*), tipe data hilang. Lebih buruk daripada JSONB. |
| Document store (misalnya MongoDB) | Cocok untuk dokumen bebas skema | Menambah satu teknologi baru. Menulis hazard dan outbox secara atomik membutuhkan transaksi lintas dokumen (perlu *replica set*), lebih berat untuk satu instance di Compose. |
| Key-value (misalnya Redis) | Sangat cepat | Tidak ada query berurutan dan paginasi berbasis kunci yang kami butuhkan, dan tidak ada transaksi hazard-plus-outbox. |
| **PostgreSQL dengan JSONB (dipilih)** | Kolom tetap untuk yang pasti, JSONB untuk yang berubah; transaksi ACID; indeks dan query lengkap; satu teknologi untuk hazard dan outbox | Lihat konsekuensi di bawah. |

**Tabel 5.5:** Perbandingan alternatif penyimpanan.

## 5.3 Batasan yang disadari

- **Demo *rebuild* memakai `pvmbg-mock`, bukan Aggregator.** Mock itu dipilih karena paling aman untuk dimatikan saat demo berjalan. Membangun ulang Aggregator berarti polling berhenti sebentar; itu tidak diperagakan oleh skrip ini.
- **Saat PVMBG dibangun ulang, data vulkanik tidak bertambah** sampai polling pulih. Data seismik dan record lama tetap tersedia dari Canonical Store.
- **Aggregator masih menggabungkan beberapa tugas** (polling, pemetaan, kepemilikan store, relay, API internal). Ini keputusan sadar (alasannya di atas), tetapi berarti satu container itu relatif besar.
- **`pkg/obs` dipakai bersama** (log, correlation ID, *health*). Isinya hanya utilitas teknis, tanpa logika bisnis, sehingga sesuai aturan soal. Namun perubahan di sana tetap berarti semua service perlu dibangun ulang agar memakainya.
- **Satu instance** PostgreSQL dan satu broker Kafka, tanpa replika. Ketahanan terhadap kegagalan bersamaan adalah bahan M2.
- Port API internal Aggregator dibuka ke *loopback* host (127.0.0.1:8083) untuk operator dan skrip demo; klien tidak memakainya.
- Karantina disimpan di memori, bukan di Canonical Store.

## 5.4 Bukti penyelesaian

Tabel berikut menjawab tiap kriteria di soal beserta status buktinya saat ini.

### Isolasi Canonical Store (`make demo-isolation`)

```
bnpb_app-net:       aggregator auth-service bmkg-mock client-api consumer-dashboard consumer-
    notifier pvmbg-mock
bnpb_canonical-net: aggregator postgres-canonical
```

| Kriteria soal | Bukti | Status |
|---|---|---|
| Setiap komponen berjalan sebagai container terpisah | Setiap service punya `Dockerfile` sendiri dan satu perintah (`docker compose up --build`) menyalakan semuanya. Daftar container ada di Tabel 1.3. | **Lulus** |
| Satu container dapat dihentikan, dibangun ulang, dan dijalankan ulang tanpa merestart yang lain; sistem tetap melayani permintaan lain | `make demo-p4`, langkah A: `pvmbg-mock` dihentikan, dibangun ulang (`docker compose build`), lalu dijalankan lagi dengan `docker compose up -d --no-deps`. Selama itu permintaan khusus data BMKG lewat Client-Facing API tetap berstatus 200. ID container dan waktu start delapan container lain dicatat sebelum dan sesudah, dan harus identik. | **Lulus** |
| HazardEvent tersimpan tanpa migrasi saat field baru muncul; record lama dan baru berdampingan | `make demo-p4`, langkah B sampai D. Laporan lama (tanpa `confidence_level`) disimpan, skema PVMBG diubah saat sistem berjalan, lalu laporan baru disimpan. Keduanya ada di PostgreSQL dan terbaca lewat Aggregator, dan atribut laporan lama tidak berubah. Sidik jari kolom tabel `hazards` sama sebelum dan sesudah, begitu pula waktu start Aggregator, jadi tidak ada DDL dan tidak ada restart. Demo Problem 1 menunjukkan hal serupa pada skala lebih besar (5 hazard dengan dan 28 tanpa field itu). | **Lulus** |
| Canonical Store hanya dapat diakses langsung oleh Aggregator | `make demo-isolation`, juga dijalankan sebagai langkah E `make demo-p4`: lihat cuplikan di bawah. | **Lulus** |

**Tabel 5.6:** Kriteria Problem 4 dan buktinya.

```
bnpb_broker-net:    aggregator consumer-dashboard consumer-notifier kafka

   PASS canonical-net contains exactly: aggregator + postgres-canonical
   PASS postgres publishes no port to the host
   PASS container on app-net (where mocks / client-facing API live): postgres-canonical:5432 is
    unreachable
   PASS container on broker-net (consumers): postgres-canonical:5432 is unreachable
   PASS control: a container on canonical-net (i.e. the Aggregator's network) does reach it
   PASS container on app-net: kafka:9092 is unreachable
   PASS the Aggregator API demands its service key (401 without, 403 with a wrong one)
ISOLATION PASS
```

Baris "control" penting: ia membuktikan bahwa penolakan di atas *bukan* karena pengujiannya rusak. Container percobaan di jaringan yang sama dengan Aggregator berhasil menyambung, sedangkan container di jaringan lain tidak.

### Field baru tanpa migrasi (`make demo-p4`, langkah B–D)

```
[2] B. Store an old-schema volcanic report
    old row attributes: {"eruption_count_24h": 3, "ash_column_height_m": 2730}
    hazards table column fingerprint before: 4555805e1c5790a6fff3c9ce2923df04
    PASS pvmbg-VR-0c2b58-0025 stored without confidence_level in PostgreSQL JSONB
[3] C. Change PVMBG's schema live and store a new report without a migration
    new row attributes: {"confidence_level": 0.82, "eruption_count_24h": 15,
                         "ash_column_height_m": 1822.6}
    hazards table column fingerprint after: 4555805e1c5790a6fff3c9ce2923df04
    PASS pvmbg-VR-0c2b58-0028 contains confidence_level; table columns and
         Aggregator start time did not change
[4] D. Read old and new records side by side through the Aggregator
    PASS old=VR-0c2b58-0025 and new=VR-0c2b58-0028 coexist and are readable;
         old attributes are unchanged
```

Sidik jari yang sama sebelum dan sesudah berarti daftar kolom tabel tidak berubah. Field baru hanya muncul di dalam kolom `attributes`.

**Membangun ulang satu service (`make demo-p4`, langkah A)**

```
[1] A. Rebuild only PVMBG while unrelated services keep serving requests
    unrelated containers before rebuild:  (8 baris: nama | ID container | waktu start)
    unrelated containers after rebuild:   (8 baris, identik)
    PASS BMKG-only requests succeeded; IDs and start times of all other
         containers stayed fixed
```

## 5.5 Jawaban butir "Di laporan"

**Alasan penentuan batas service**

Batas ditarik menurut *alasan berubah* dan *kepemilikan data*. Sumber, Aggregator (pemilik Canonical Store), autentikasi, penyajian, dan consumer dipisah karena masing-masing berubah karena alasan yang berbeda. Polling, pemetaan, penyimpanan, dan outbox tetap satu container karena hazard dan baris outbox harus ditulis dalam satu transaksi. Pembagian per lapisan teknis ditolak karena mengubah transaksi itu menjadi terdistribusi; pembagian per sumber adalah kandidat M2. Rincian di bagian 5.2.1.

**Perbandingan alternatif penyimpanan**

Dibandingkan: relasional kolom tetap, EAV, document store, key-value, dan PostgreSQL dengan JSONB (bagian 5.2.2). PostgreSQL dengan JSONB dipilih karena memberi fleksibilitas dokumen untuk field yang berubah *sekaligus* transaksi ACID yang dibutuhkan pola outbox, dengan satu teknologi saja. Konsekuensinya: isi `attributes` belum divalidasi tipenya dan belum diindeks.

# 6 Problem 5 – Fan-out Event Distribution (Pub/Sub)

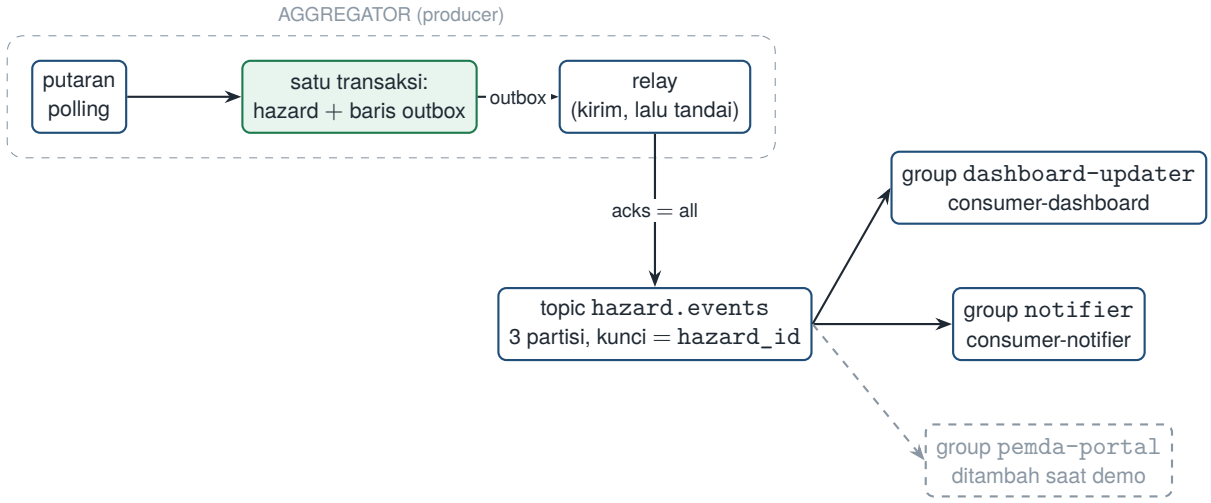
**Singkatnya**

Aggregator **tidak pernah memanggil consumer**. Setiap perubahan hazard ditulis ke tabel *outbox* dalam transaksi yang sama dengan hazard-nya, lalu sebuah *relay* menerbitkannya ke topic Kafka `hazard.events`. Consumer berlangganan sendiri dengan *consumer group* masing-masing. Pengiriman **at-least-once**; karena itu consumer membuang duplikat berdasarkan `event_id`. Consumer yang mati tidak menahan producer dan akan menyusul event yang terlewat saat hidup lagi.

**Status bukti Problem 5**

Keempat kriteria terbukti lewat satu skrip, `make demo-p5`: event lewat broker, dua consumer independen, consumer mati lalu menyusul *dengan ID event yang tepat*, dan consumer ketiga (`pemda-portal`) yang ditambahkan saat sistem berjalan tanpa menyentuh Aggregator. Consumer ketiga ini tidak ada di Compose; ia dinyalakan oleh skrip demo untuk membuktikan bahwa menambah consumer tidak memerlukan perubahan di producer.

## 6.1 Rancangan solusi dan mekanisme




**Gambar 6.1:** Alur event. Producer hanya mengenal topic; consumer tidak dikenal producer. Garis putus-putus: consumer yang ditambahkan saat demo tanpa mengubah producer.

### 6.1.1 Sisi producer: transactional outbox

1. Saat Aggregator menyimpan hazard baru atau versi baru, ia menulis *juga* satu baris `outbox` dalam **transaksi database yang sama**. Keduanya berhasil bersama atau gagal bersama, sehingga tidak ada hazard yang tersimpan tanpa event.
2. **Relay** (`aggregator/internal/relay`) membaca baris outbox yang belum terkirim, mengirimnya ke Kafka berurutan dari yang paling lama, dan baru *setelah* Kafka mengakui menandainya terkirim. Bila satu pengiriman gagal, relay berhenti di situ (menjaga urutan per hazard), menunggu dengan jeda yang makin panjang, lalu mencoba lagi.

3. Akibatnya: **Kafka yang mati tidak menghentikan ingest dan tidak menghilangkan event.** Event menunggu di outbox dan terkirim berurutan setelah broker pulih (`TestBrokerOutageDoesNotStopIngestionOrLoseEvents`).
4. Producer Kafka memakai `acks=all` dan mode idempoten (`franz-go`).

| Item | Nilai |
|---|---|
| Topic | `hazard.events`: 3 partisi, replikasi 1, retensi 7 hari. Dibuat oleh `kafka-init`; pembuatan otomatis dimatikan. |
| Kunci pesan | `hazard_id`: semua versi satu hazard masuk satu partisi, jadi urutannya per hazard terjaga. |
| Isi pesan | JSON: `event_id`, `kind` (`created` atau `updated`), `emitted_at`, `hazard`, `schema`. Skemanya di `contracts/schemas/kafka-envelope.json`. |
| Header pesan | `correlation_id` (sama dengan siklus polling yang menghasilkan perubahan) dan `event_id`. |
| Kunci idempotensi | `event_id = <hazard_id>:<version>` |
| Consumer group | Satu group per consumer: `dashboard-updater` dan `notifier`; `pemda-portal` ditambahkan saat demo. |

**Tabel 6.1:** Konfigurasi Kafka dan format pesan.

### 6.1.2 Sisi consumer

Setiap consumer (`services/consumer-hazard`) berlangganan topic dengan group-nya sendiri. Untuk tiap pesan ia (1) membaca header untuk melanjutkan `cid`, (2) membuang pesan yang `event_id`-nya sudah pernah diproses, (3) memproses, lalu (4) menandai *offset* untuk di-commit. Karena offset ditandai *setelah* diproses, pesan tidak pernah hilang bila consumer mati di tengah; paling buruk pesan itu diproses lagi.

### 6.1.3 Semantik pengiriman: at-least-once

- Producer → Kafka: relay menandai baris terkirim hanya setelah mendapat pengakuan. Bila Aggregator mati *tepat di antara* pengakuan dan penandaan, baris yang sama dikirim ulang. Satu event bisa tiba dua kali, tidak pernah nol kali.
- Kafka → consumer: offset di-commit setelah pemrosesan, jadi pesan yang belum selesai diproses akan dikirim ulang setelah consumer hidup kembali.
- **Konsekuensinya, consumer wajib idempoten.** Kelompok menanganinya dengan membuang duplikat berdasarkan `event_id`, yang bersifat deterministik (`hazard_id:version`).

## 6.2 Alasan keputusan dan trade-off

Tabel berikut membandingkan beberapa cara menyebarkan event dengan pilihan kami.

**Pola:** *publish-subscribe* dengan satu consumer group per pihak berlangganan. Bila dua consumer dimasukkan ke group yang sama, Kafka akan *membagi* event di antara mereka (pola antrean kerja), bukan menyalinnya. Karena tiap pihak harus menerima *semua* event, kami memberi tiap pihak group sendiri.

**Semantik:** *at-most-once* ditolak karena event peringatan bisa hilang diam-diam, dan itu tidak dapat diterima untuk data kebencanaan. *Exactly-once* ditolak karena menuntut transaksi dari ujung ke ujung dan jauh lebih rumit; consumer yang idempoten memberi hasil efektif yang sama dengan biaya jauh lebih kecil.

| Alternatif | Pertimbangan |
|---|---|
| Producer memanggil tiap consumer langsung (HTTP) | Persis masalah di soal: consumer lambat menahan producer, dan consumer baru berarti mengubah kode producer. |
| RabbitMQ | Antrean yang matang dan ringan. Pada antrean biasa, pesan yang sudah dikonsumsi dihapus, sehingga consumer baru tidak bisa membaca ulang dan setiap consumer butuh antreannya sendiri yang harus dikelola. (RabbitMQ Streams menutup celah ini, tetapi itu pola lain yang tidak kami telusuri.) |
| NATS (core) | Sangat ringan dan cepat, tetapi tanpa penyimpanan pesan; consumer yang mati kehilangan event, bertentangan dengan kriteria menyusul. (JetStream menambahkan penyimpanan, tetapi tidak kami pilih.) |
| Redis Pub/Sub | Sederhana, tetapi *fire-and-forget*: consumer yang mati tidak menerima apa pun saat kembali. |
| **Apache Kafka (dipilih)** | Log yang tahan lama: pesan disimpan sesuai retensi dan tiap consumer group punya *offset* sendiri. Consumer yang mati menyusul dari offset terakhirnya, dan group baru dapat berlangganan tanpa menyentuh producer. |

**Tabel 6.2:** Alternatif broker dan pola distribusi.

**Harga yang diterima.** Kafka lebih berat dibanding RabbitMQ atau NATS (membutuhkan memori JVM; heap dibatasi 256–512 MB di Compose) dan operasinya lebih rumit. Untuk kebutuhan menyusul setelah mati dan consumer baru tanpa mengubah producer, harga itu dinilai layak.

## 6.3 Batasan yang disadari

- **Pembuangan duplikat hanya di memori.** Himpunan `event_id` hilang saat consumer direstart, dan tidak ada penghapusan entri lama (memori tumbuh seiring jumlah event). Setelah restart, pesan yang belum di-commit bisa diproses lagi. Pada M1 consumer hanya mencatat dan menghitung, sehingga aman; untuk aksi nyata (misalnya notifikasi sungguhan) pemrosesan ulang harus aman diulang atau memakai penyimpanan dedup yang tahan lama.
- **Retensi 7 hari.** Consumer yang mati lebih lama dari itu kehilangan event yang sudah kedaluwarsa.
- **Group baru membaca semua riwayat yang masih tersimpan.** Kami mengecek ini secara terpisah dari skrip demo: group baru memproses 153 event, sama dengan jumlah di dashboard. Berguna untuk consumer yang perlu mengejar data, tetapi consumer yang hanya ingin event baru harus mengaturnya sendiri.
- **Satu broker, replikasi 1.** Bila disk broker hilang, event yang belum dibaca ikut hilang. Replikasi dan penanganan kegagalan bersamaan adalah bahan M2.
- **Urutan hanya terjamin per hazard** (satu kunci, satu partisi), bukan urutan global lintas hazard.
- **Pesan rusak dilewati** dan dicatat di log (`consumer_bad_message`); belum ada *dead-letter queue*.
- **Consumer pada M1 bersifat tiruan:** dashboard dan notifier hanya mencatat dan menghitung event, belum menggerakkan tampilan atau mengirim notifikasi sungguhan.
- **Satu relay** per Aggregator. Bila Aggregator dijalankan banyak instance (M2), pembacaan outbox perlu dikunci agar tidak terkirim ganda.
- Consumer untuk portal pemerintah daerah (`pemda-portal`) hanya ada saat demo. Ia tidak termasuk dalam Compose sehari-hari.

## 6.4 Bukti penyelesaian

Bukti berasal dari `make demo-p5` (transkrip: `docs/evidence/p5/latest.log`) dan dari tes otomatis di `aggregator/internal/relay`.

| Kriteria soal | Bukti | Status |
|---|---|---|
| Event dipublikasikan ke broker, bukan langsung ke consumer | Alur outbox dan relay. Skrip demo menunggu log `outbox_published` untuk ID event yang baru dibuat, yaitu tanda bahwa Kafka sudah mengakuinya. Tes di `relay_test.go` (daftar di bawah tabel). Aggregator tidak memiliki alamat consumer mana pun di konfigurasinya. | **Lulus** |
| Dua consumer independen menerima event, producer tidak mengenal mereka | Langkah B: satu event BMKG baru dibangkitkan, dan kedua consumer (group berbeda) sama-sama mencatat ID yang sama (`bmkg-EQ-9b73f8-0061:1`) di lognya. | **Lulus** |
| Satu consumer dihentikan: producer tidak tertahan, consumer lain normal; setelah hidup lagi, apa yang terjadi pada event yang terlewat | Langkah C: `consumer-notifier` dihentikan; Aggregator tetap menyimpan dan menerbitkan event baru (`bmkg-EQ-9b73f8-0063:1`), dan dashboard tetap menerimanya. Langkah D: notifier dinyalakan lagi dan **menyusul** event yang tepat itu. Skrip memeriksa ID event di log notifier, bukan sekadar kenaikan angka. Event tidak hilang karena Kafka menyimpannya dan offset group dicatat di broker. | **Lulus** |
| Menambah consumer baru pada topic yang sudah ada, tanpa mengubah kode producer | Langkah E: container baru dengan `KAFKA_GROUP=pemda-portal` dinyalakan lewat `docker compose run` saat sistem berjalan. Event berikutnya (`bmkg-EQ-9b73f8-0065:1`) diterima ketiga group. Skrip memeriksa bahwa ID container dan waktu start Aggregator tidak berubah. | **Lulus** |

**Tabel 6.3:** Kriteria Problem 5 dan buktinya.

### Cuplikan transkrip (`make demo-p5`)

```
[1] A. Both consumers are healthy and subscribed
    PASS consumer-dashboard /health        PASS consumer-notifier /health
[2] B. New hazard event is published and received by BOTH consumers
    generated BMKG event EQ-9b73f8-0061 - waiting for Kafka fan-out
    PASS dashboard processed=126 notifier processed=23
[3] C. Notifier stopped - Aggregator still ingests; dashboard still receives events
    PASS aggregator ingested bmkg-EQ-9b73f8-0063 while notifier is down
    PASS dashboard still consuming (producer not blocked by dead consumer)
[4] D. Notifier restarts and catches up
    PASS notifier caught up with the exact offline event bmkg-EQ-9b73f8-0063:1
[5] E. Add a new consumer group to the same topic without touching the producer
    PASS dashboard, notifier, and new pemda group received bmkg-EQ-9b73f8-0065:1;
         producer was unchanged
    PASS P5 pub/sub demo complete
```

Angka `processed` berbeda antara dashboard dan notifier karena skrip me-restart notifier, dan penghitungnya disimpan di memori sehingga mulai dari nol lagi. Karena itu skrip membuktikan ID event, bukan jumlahnya.

**Tes otomatis terkait** (`relay_test.go`):

- `TestFlushPublishesInOrderWithHeadersAndMarksDelivered`: urutan, header, dan penandaan terkirim.

- `TestBrokerOutageDoesNotStopIngestionOrLoseEvents`: broker mati tidak menghentikan ingest dan tidak menghilangkan event.
- `TestPartialFailureKeepsOrderAndResumes`: gagal di tengah batch menjaga urutan dan melanjutkan.
- `TestRunDeliversNewEntriesUntilCancelled`: relay terus mengirim entri baru sampai dihentikan.

## 6.5 Jawaban butir "Di laporan"

**Broker dan pola yang dipilih**

**Apache Kafka** (mode KRaft, satu broker) dengan pola *publish-subscribe*: satu topic `hazard.events`, satu consumer group per pihak yang berlangganan. Dipilih karena pesannya disimpan sehingga consumer yang mati dapat menyusul, dan group baru dapat ditambahkan tanpa mengubah producer. Producer menulis lewat *transactional outbox* agar tidak ada event hilang bila broker sedang mati.

**Semantik pengiriman dan konsekuensi bagi consumer**

**At-least-once.** Producer menunggu pengakuan penuh (`acks=all`, idempoten), relay menandai terkirim setelah pengakuan, dan consumer meng-commit offset setelah memproses. Event tidak hilang, tetapi dapat tiba lebih dari sekali. Karena itu **consumer harus idempoten**. Kelompok menyikapinya dengan membuang duplikat berdasarkan `event_id = hazard_id:version`, yang deterministik. Batasannya: himpunan dedup berada di memori dan hilang saat restart, sehingga aksi sungguhan pada consumer kelak harus aman diulang.

# A Penelusuran Klaim ke Bukti

Soal meminta setiap klaim dapat ditelusuri ke bukti atau ke kode. Lampiran ini merangkum jalurnya. Semua demo memeriksa dirinya sendiri dan menyimpan transkrip ke `docs/evidence/`.

## A.1 Ketentuan implementasi lintas problem

| Ketentuan di soal | Cara memeriksa | Kode atau bukti |
|---|---|---|
| Komunikasi lewat jaringan; tidak berbagi library business-logic | `pkg/obs` hanya utilitas teknis; tiap service punya `go.mod` sendiri | `pkg/obs`, `services/*/go.mod` |
| Satu storage satu pemilik; Canonical Store hanya diakses Aggregator | `make demo-isolation` | `docs/evidence/p4/isolation.log` |
| Satu container per service; satu perintah orkestrasi | `make reset` atau `docker compose up --build` | `services/*/Dockerfile`, `docker-compose.yml` |
| Kredensial lembaga tidak berlaku lintas instansi | `make demo-mock-auth` | `docs/evidence/p3/mock-auth.log` |
| Tidak ada kredensial di kode atau commit; `.env.example` tersedia | `make secrets` | `.env.example`, `.gitignore`, `.gitleaks.toml` |
| `/health`, log terstruktur ber-`cid`, latensi panggilan keluar | Langkah 8 demo Problem 1; `cid-trace.log` Problem 2 | `pkg/obs` |
| Polling dari BNPB, interval dapat diatur, sedikitnya satu event per 10 detik, seed 20 record | Tes mock; demo Problem 1 | `services/*-mock/internal/*/generator.go` |
| Kegagalan benar-benar dipicu saat demo | `make demo-p2-aggregator`, `make demo-p2` | `POST /admin/outage`, `POST /admin/delay` |

**Tabel A.1:** Ketentuan implementasi dan cara memeriksanya.

## A.2 Matriks kriteria, perintah, dan bukti

| Problem | Perintah | Berkas bukti | Status |
|---|---|---|---|
| P1 | `make demo-p1` | `docs/evidence/p1/latest.log` | **Lulus** |
| P2 (ingest) | `make demo-p2-aggregator` | `docs/evidence/p2/aggregator-latest.log` | **Lulus** |

| Problem | Perintah | Berkas bukti | Status |
|---|---|---|---|
| P2 (layanan, load test) | `make demo-p2` | `docs/evidence/p2/latest.log`, `k6-summary.json`, `k6-overload-summary.json`, `cid-trace.log` | **Lulus** |
| P3 (kredensial mock) | `make demo-mock-auth` | `docs/evidence/p3/mock-auth.log` | **Lulus** |
| P3 (klien downstream) | `make demo-p3` | `docs/evidence/p3/latest.log` | **Lulus** |
| P3 (tanpa kredensial ter-commit) | `make secrets` | keluaran `scripts/check-secrets.sh` | **Lulus** |
| P4 (isolasi storage) | `make demo-isolation` | `docs/evidence/p4/isolation.log` | **Lulus** |
| P4 (JSONB, record berdampingan) | `make demo-p4` (langkah B–D) | `docs/evidence/p4/latest.log` | **Lulus** |
| P4 (hentikan, bangun ulang, jalankan ulang) | `make demo-p4` (langkah A) | `docs/evidence/p4/latest.log` | **Lulus** |
| P5 (fan-out, consumer mati, menyusul) | `make demo-p5` | `docs/evidence/p5/latest.log` | **Lulus** |
| P5 (consumer baru tanpa mengubah producer) | `make demo-p5` (langkah E) | `docs/evidence/p5/latest.log` | **Lulus** |

## A.3 Perintah pemeriksaan penuh

```
make lint && make test-race                                    # kualitas kode dan tes (dengan
    race detector)
make reset && make demo-p1 demo-p2-aggregator demo-mock-auth demo-isolation
make demo-p4 demo-p5                                           # rebuild satu service, field baru
     tanpa migrasi, consumer baru
```

```
make demo-p2 demo-p3                                          # termasuk load test dan menunggu
    token kedaluwarsa
make secrets                                                  # sebelum tag milestone-1
```

# B Lampiran Form Penggunaan AI

**Diisi oleh kelompok**

Halaman ini disediakan untuk *Form Penggunaan AI.pdf* yang wajib dilampirkan oleh semua kelompok. **[lampirkan formulir yang sudah diisi]**